

Software Engineering Body of Knowledge (SWEBOK)

Rodrigo Martins Pagliares

Software Engineering Body of Knowledge (SWEBOK)

SWEBOK

Software Engineering Course (x +

computer.org/education/bodies-of-knowledge/software-engineering

☆

□

👤

⋮

Home / Education / Bodies of Knowledge

Software Engineering Body of Knowledge (SWEBOK)

DOWNLOAD

Search this page...

Page Content:

Alert

About SWEBOK

An Evolving Body of Knowledge

Objectives

Who Benefits

SWEBOK Overview

Volunteer

Citation Information

Alert

If you are interested in the SWEBOK V4 public review form, please visit our webpage here:
<https://www.computer.org/volunteering/boards-and-committees/professional-educational-activities/software-engineering-committee/swebok-evolution>



About SWEBOK

The Guide to the Software Engineering Body of Knowledge (SWEBOK) is a comprehensive resource for software engineering. Its 15 knowledge areas (KAs) summarize key information.

https://bit.ly/3BLn4dN

Knowledge Area – KA

Software requirements

Introduction

The two software requirements perspectives

1. As an **expression** of the **needs** and **constraints** on a **software product** or **project** that **contribute** to the **solution** of a **real-world problem**.
2. The **activities** necessary to **develop** and **maintain** the **requirements** for a **software product** and for the **project** that **constructs** it.

**Both perspectives are presented
in this knowledge area (KA)**

The importance of requirements for success

- **Industry experts and researchers generally agree on the importance of software requirements for success.**
- **If a team does a poor job of determining the requirements, the project, the product or both are likely to suffer from:**
 - ✓ **Added costs**
 - ✓ **Delays**
 - ✓ **Cancellations**
 - ✓ **Defects.**

Why poor requirements does affect a project?

- Each software product requirement generally leads to many design decisions.
- Each design decision generally leads to many code-level decisions.
- Each decision can involve several test decisions.

Why poor requirements does affect a project?

- In other words, **determining the requirements correctly is high-stakes work.**
- **If not detected and repaired early, missing, misinterpreted and incorrect requirements can induce exponentially cascading rework to correct them.**

Two primary requirements-related problems

- Software projects tend to suffer from two primary requirements-related problems:

1. Incompleteness

- ✓ Stakeholder requirements exist that are not revealed and communicated to the software engineers.

2. Ambiguity

- ✓ Requirements are communicated in a way that is open to multiple interpretations, with only one possible interpretation being correct.

Bug & defect

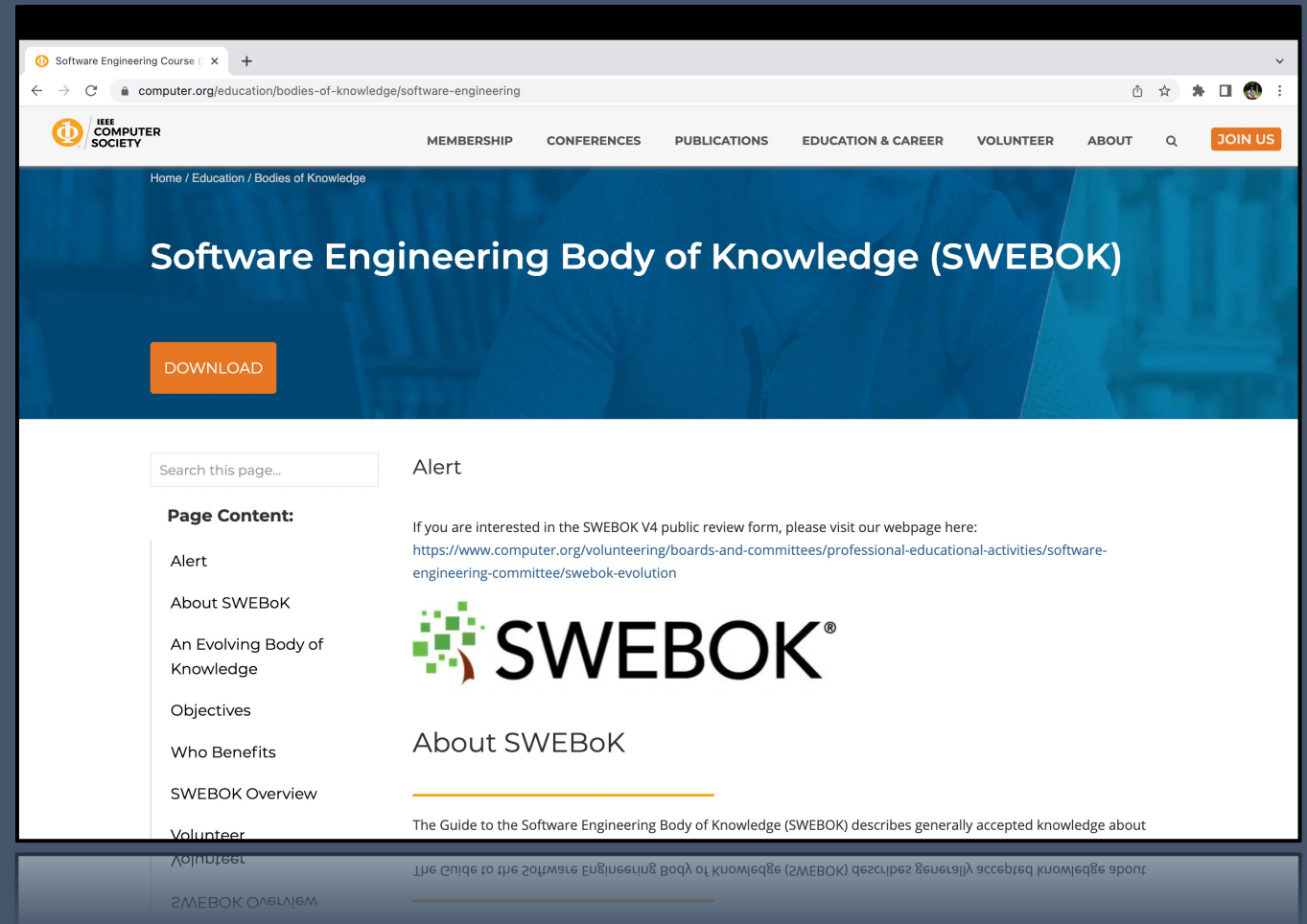
- The **challenge** is **determining WHAT** that **code** is **intended** to do.
- A **bug** is an **observable difference between** what the **software** is **intended to do** and **what it does**.
 - ✓ Some authors prefer the name **defect** instead of **bug**.

The role of requirements documentation

- To capture and communicate intent for software engineers who maintain the code but might not have been its original authors.
- Throughout the service life of the software.

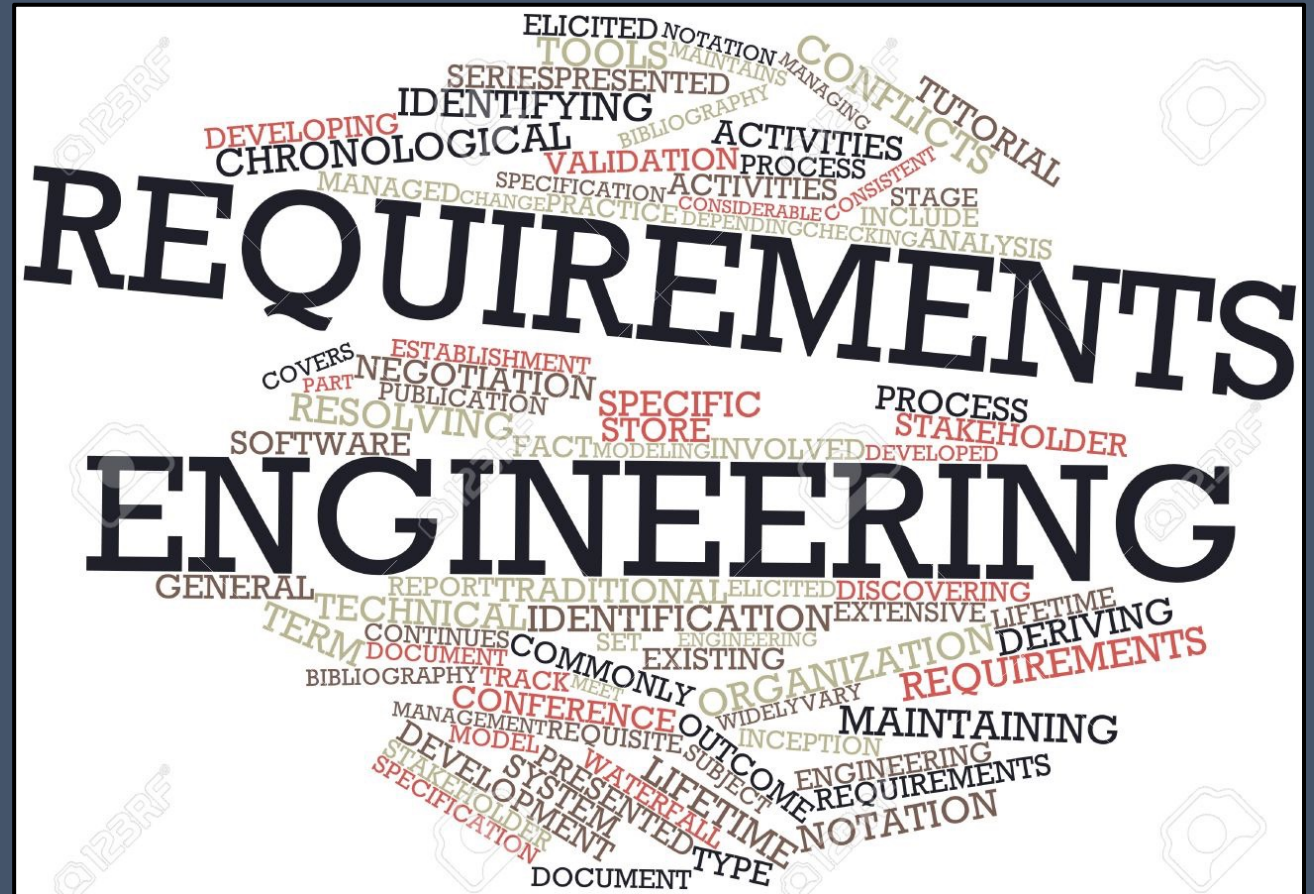
The Software Requirements KA (SWEBOK)

- Concerns developing and managing software requirements over the software's service life.



Requirements engineering

- The term requirements engineering is often used to denote the systematic handling of requirements.



Source: <https://bit.ly/3MVQco1>

Requirements with RUP and agile methods

- **Rational Unified Process (RUP)**
 - ✓ **Initial, high-level requirements work** is done during an **Inception phase**
 - ✓ **Further detailing** is done **during one or more iterations** of the **Elaboration phase**.
- **Agile methods**
 - ✓ **Requirements work** is done **incrementally, just in time**, as each **additional element of functionality** is **constructed**.

Breakdown topics for software requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification
- Additional Attributes of Requirements
- Incremental and Comprehensive Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:

1. Software Requirements fundamentals

Software Requirements

1

Software Requirements Fundamentals

- Definition of a Software Requirement **1.1**
- Categories of Software Requirements **1.2**
- Software Product Requirements and Software Project Requirements **1.3**
- Functional Requirements **1.4**
- Nonfunctional Requirements **1.5**
- Technology Constraints **1.6**
- Quality of Service Constraints **1.7**
- Why Categorize Requirements This Way? **1.8**
- System Requirements and Software Requirements **1.9**
- Derived Requirements **1.10**
- Software Requirements Activities **1.11**

Requirements Elicitation

- Requirements
- Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of

Requirements Specification

- Unstructured Natural Language Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change

Practical Considerations

- Iterative Nature of the Requirements Process

Software Requirements Tools

- Requirements Management Tools
- Requirements

Work breakdown item:
1. Software Requirements fundamentals

- Model-Based Requirements Specification
- Additional Attributes of Requirements
- Incremental and Comprehensive Requirements Specification

Work breakdown item:

1.1 Definition of a Software Requirement

Software Requirements

Software Requirements Fundamentals

Definition of a Software Requirement **1.1**

Categories of Software Requirements

- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

Requirements Sources

Requirements Analysis

Basic Requirements Analysis

Economics of Quality of Service Constraints

Formal

Requirements Specification

Unstructured Natural Language Requirements Specification

Structured Natural

Requirements Validation

Requirements Reviews

Simulation and Execution

Prototyping

Requirements Management Activities

Requirements Scrubbing

Requirements Change Control

Scope Matching

Practical Considerations

Iterative Nature of the Requirements Process

Requirements Prioritization

Requirements

Software Requirements Tools

Requirements Management Tools

Requirements Modeling Tools

Functional Requirements Case Generation Tools

Work breakdown item:

1.1. Definition of a software requirement

Additional Attributes of Requirements

Incremental and Comprehensive Requirements Specification

References used for this work breakdown item

Search Results

30 results were found. [Print Results](#)

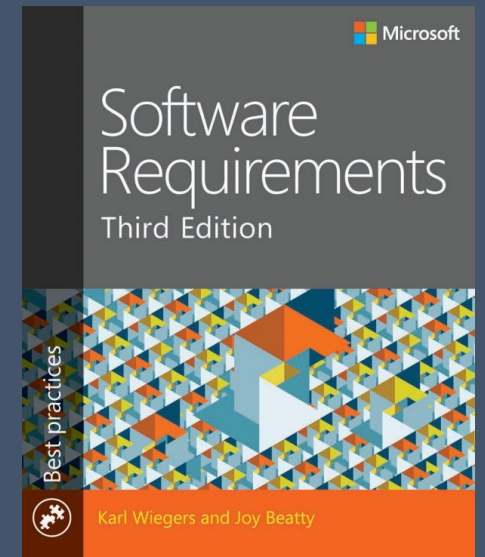
software requirement. (1) software capability needed by a user to solve a problem or to achieve an objective (*ISO/IEC/IEEE 24765:2017 Systems and software engineering-Vocabulary*) (2) software capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed document (*ISO/IEC/IEEE 24765:2017 Systems and software engineering-Vocabulary*)

software requirements specification (SRS). (1) documentation of the essential requirements (functions, performance, design constraints, and attributes) of the software and its external interfaces (*IEEE 1012-2016 IEEE Standard for System, Software, and Hardware Verification and Validation*, 3.1.29)

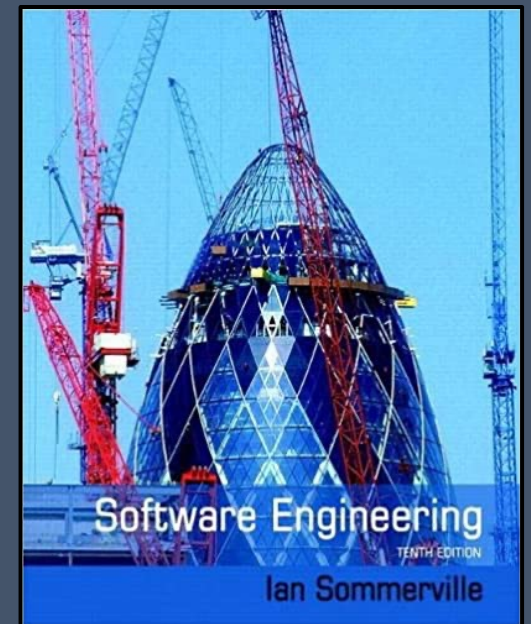
software requirements review (SRR). (1) review of the requirements specified for one or more software configuration items to evaluate their responsiveness to and interpretation of the system requirements and to determine whether they form a satisfactory basis for proceeding into preliminary design of the configuration items (*ISO/IEC/IEEE 24765:2017 Systems and software engineering-Vocabulary*) (2) review as in (1) for any software component (*ISO/IEC/IEEE 24765:2017 Systems and software engineering-Vocabulary*) Note: This review is called software specification review by the US Department of Defense. See Also: system requirements review

<https://bit.ly/3TeV6hW>

user needs to arrive at a
IEEE 24765:2017 Systems



Chapter 1, pages 5-6



Chapter 4, page 102

Definition of a software requirement

1. A **condition or capability** needed by a user to solve a **problem** or **achieve an objective**.
2. A **condition or capability** that must be **met or possessed** by a **system or system component** to **satisfy a contract, standard, specification** or other **formally imposed document**.
3. A **documented representation or capability** as in (1) or (2) above.

This formal definition is extended in this KA to include expressions of a software project's needs and constraints.

At its most basic, a software requirement is a property that must be exhibited to solve a real-world problem.

Who does impose requirements?

- Clients
- Customers
- Users
- Regulatory authorities
- The software organization
- The project itself

Work breakdown item:

1.2. Categories of Software Requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Ambiguity

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Incremental and Comprehensive Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

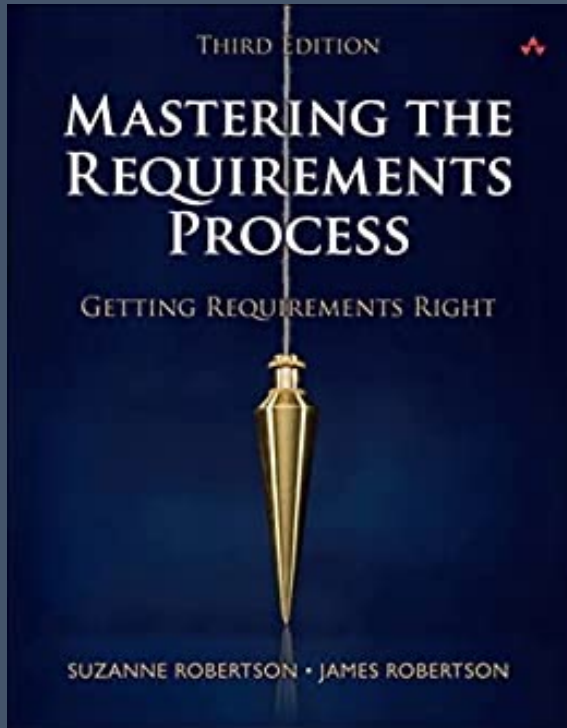
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Reuse

Software Requirements Tools

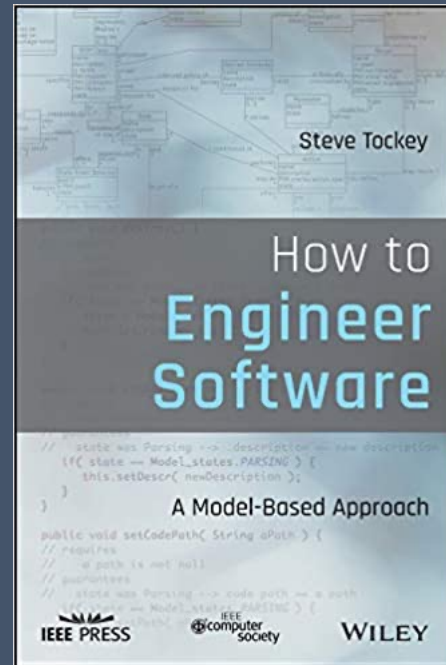
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
1.2. Categories of software requirements

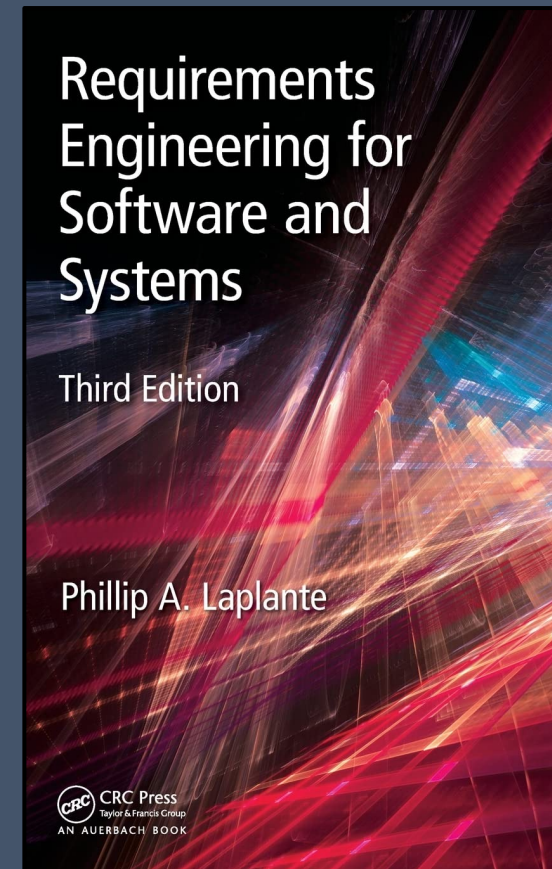
References used for this work breakdown item



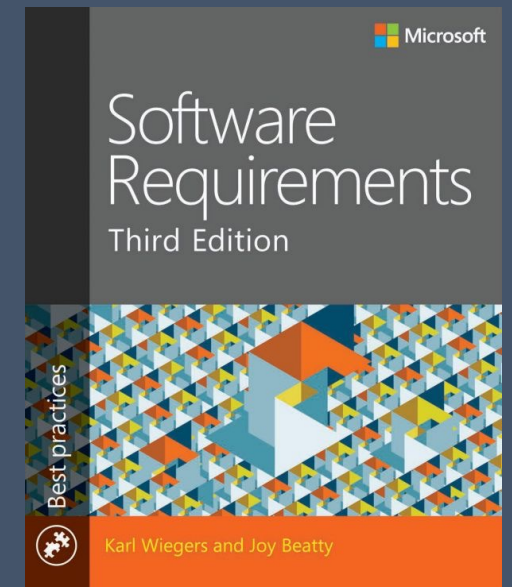
Chapter 1



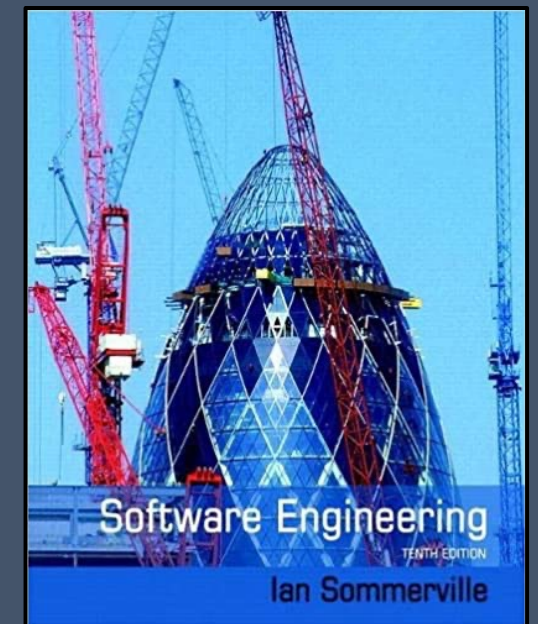
Chapter 4



Chapter 1

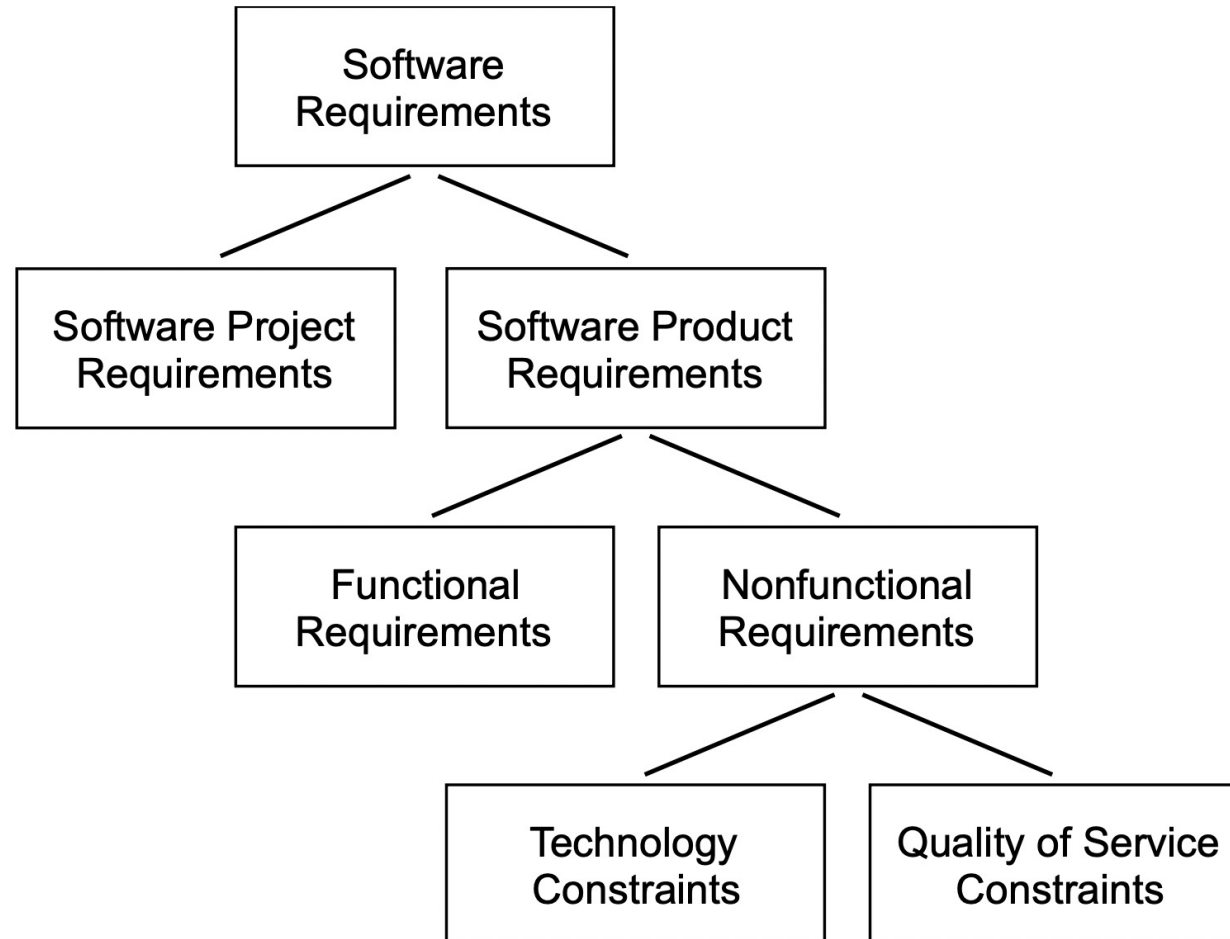


Chapter 1, pages 7-12



Section 4.1

Categories of software requirements



Work breakdown item:

**1.3. Software product requirements and
software project requirements**

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements **1.3**
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

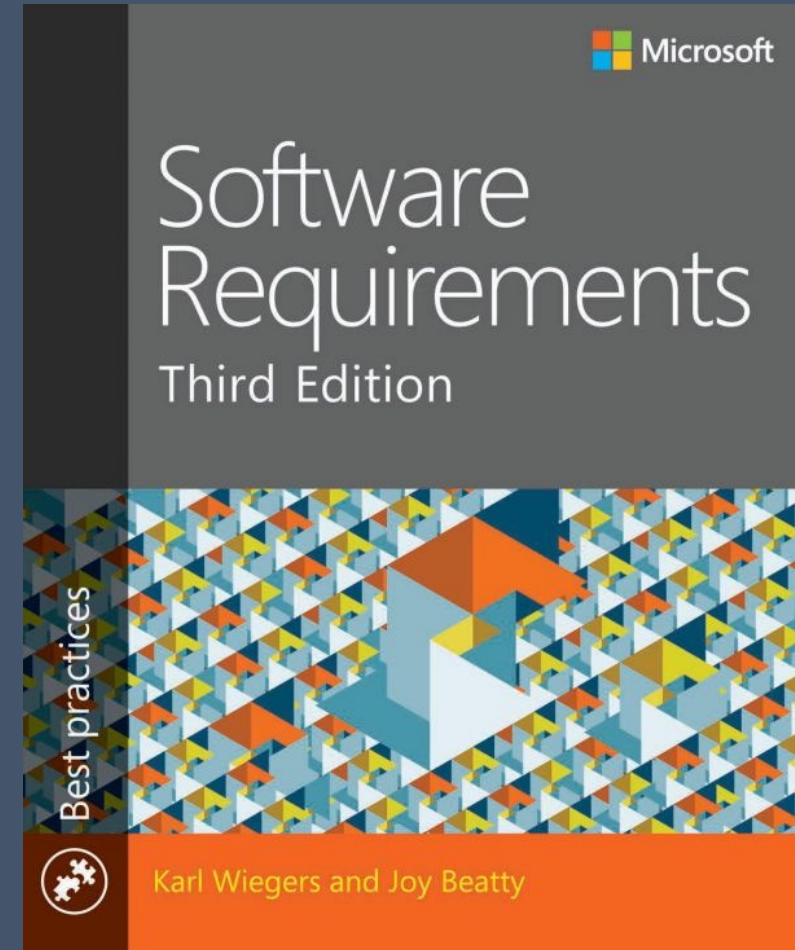
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
1.3. Software product requirements and software project requirements

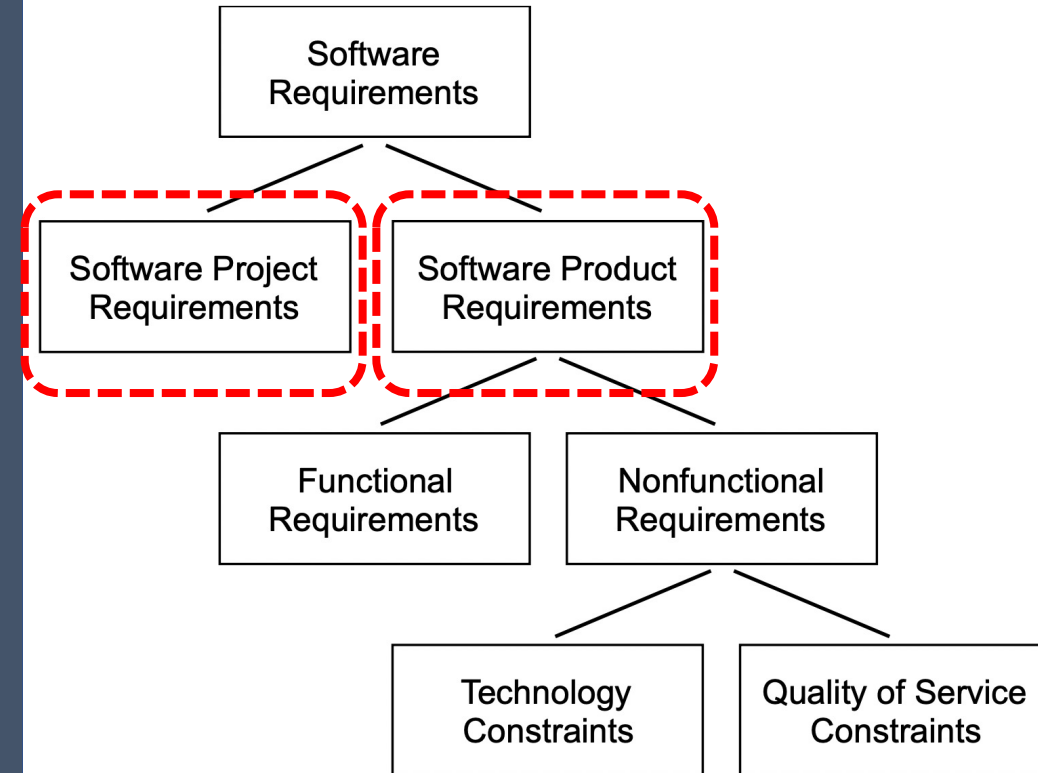
References used for this work breakdown item



Chapter 1, pages 14-15

Software requirements

- **Software product requirements**
 - ✓ Specify the software's expected form, fit or function.
- **Software project requirements**
 - ✓ Also called **process requirements**.
 - ✓ Constrain the **project** that constructs the software.



Software project requirements

- **Project requirements often constrain cost, schedule and/or staffing**
 - ✓ But can also constrain other aspects of a software project, such as maintenance.
- Can be captured in a **project charter** or other **high-level project initiation document**.
- Most relevant to **how** the **project** is **managed** or what **life cycle process** should be used.

Software project requirements

- This KA (Software Requirements) does not discuss software project requirements further.
- For more information:
 - ✓ Software Engineering Management KA.
 - ✓ Software Engineering Process KA.

Work breakdown item:

1.4. Functional requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

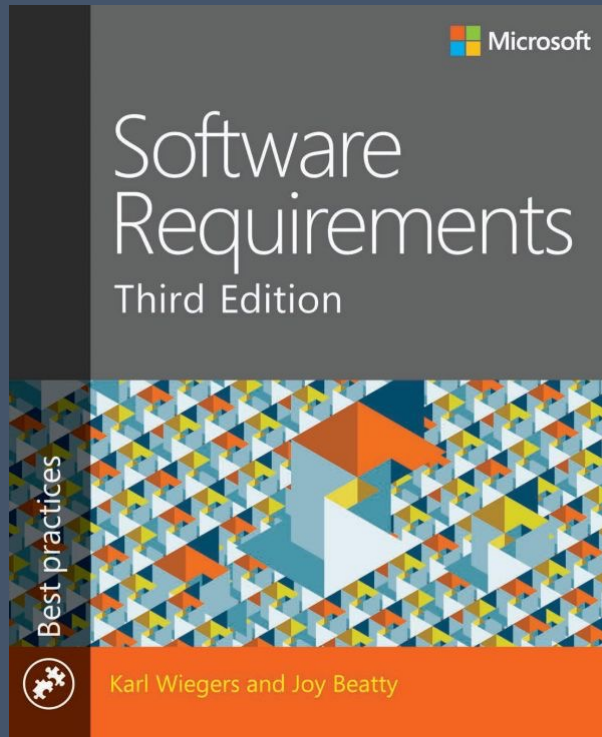
Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

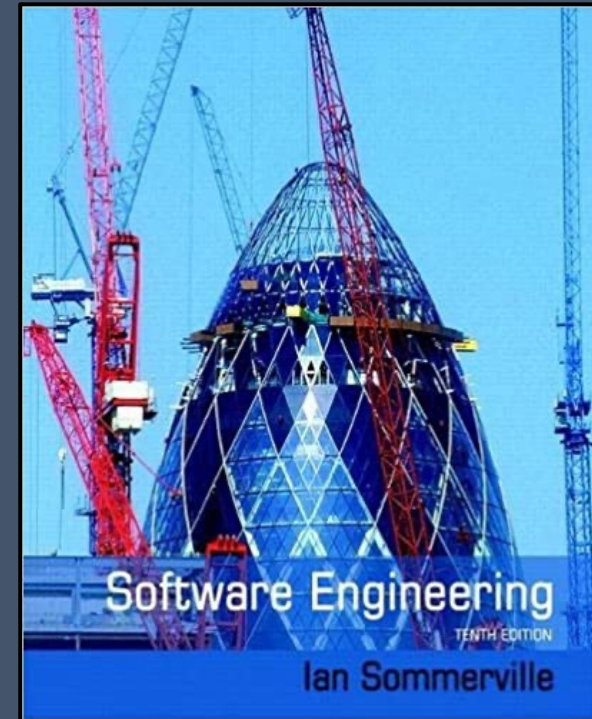
**Work breakdown item:
1.4. Functional requirements**

1.4

References used for this work breakdown item



Chapter 1, page 9



Section 4.1.1

Functional requirements

- **Specify observable behaviors that the software is to provide.**
- **Policies to be enforced and processes to be carried out.**

Work breakdown item:

1.5. Nonfunctional requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- **Nonfunctional Requirements**
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

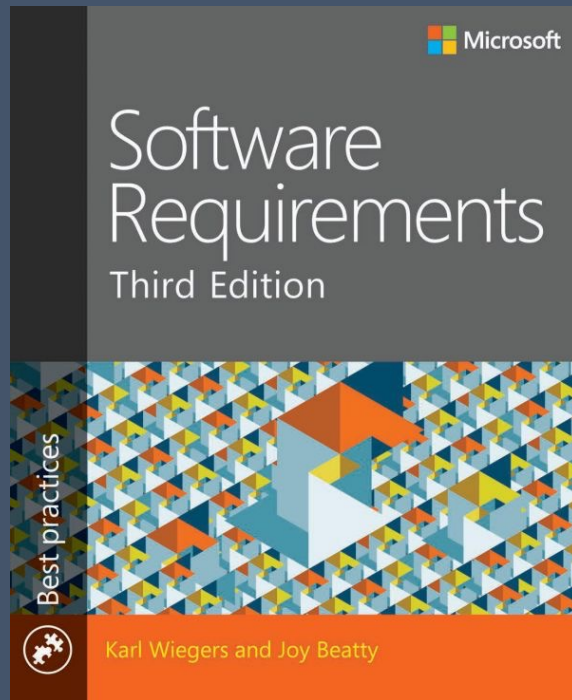
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

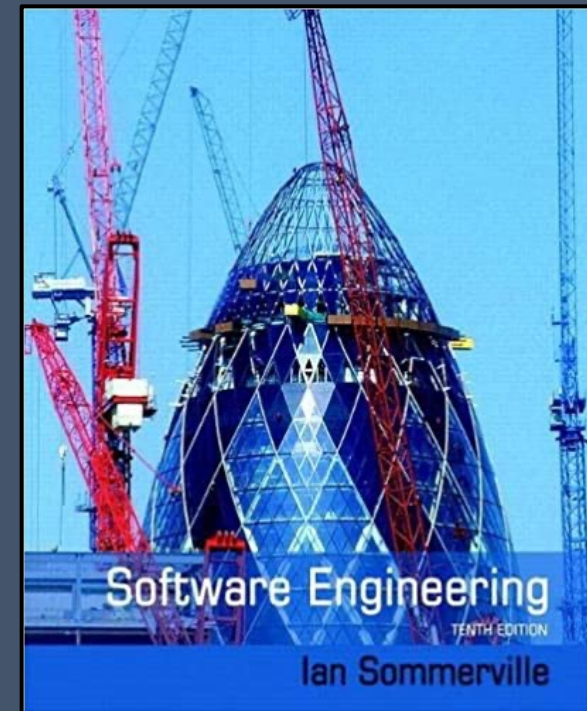
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
1.5. Nonfunctional requirements

References used for this work breakdown item



Chapter 1, pages 10-11



Section 4.1.2

Nonfunctional requirements

- **Constrain** in some way the **technologies** to be **used** in the **implementation**:
 - ✓ What **computing platform(s)**?
 - ✓ What **database engine(s)**?
 - ✓ **How accurate** do results need to be?
 - ✓ **How quickly** must results be presented?
 - ✓ **How many records** of a certain type need to be **stored**?

Nonfunctional requirements

- Some **nonfunctional requirements** might **relate** to the **operation** of the **software**.
 - ✓ See the **Operation** and **Maintenance** KA.

Nonfunctional requirements

- The **nonfunctional requirements** can be further **divided**:
 - ✓ **Technology constraints**
 - ✓ **Quality of service constraints.**

Work breakdown item:

1.6. Technology constraints

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

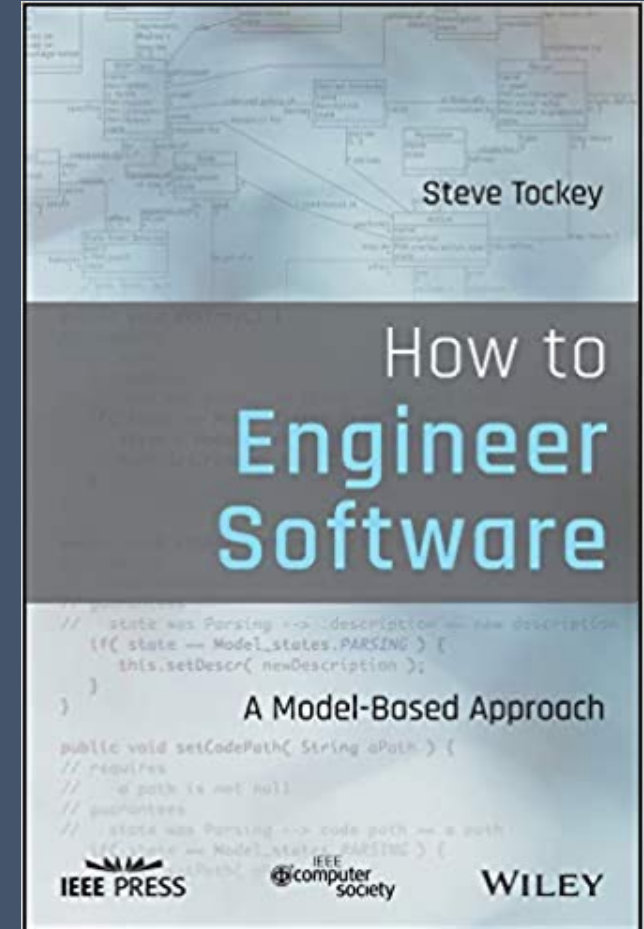
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
1.6. Technology constraints

References used for this work breakdown item



Chapter 4

Technology constraints

- **Mandate or prohibit** use of specific, named automation **technologies** or defined **infrastructures**.

Examples of technology constraints

- **Computing platforms:**
 - ✓ Windows, MacOS, Android OS, iOS.
- **Programming languages:**
 - ✓ Java, C++, C#, Python.
- **Compatibility with specific web browsers:**
 - ✓ Chrome, Safari, Edge.
- **Database engines:**
 - ✓ Oracle, SQL Server, MySQL.
- **General technologies :**
 - ✓ Reduced Instruction Set Computer (**RISC**), **Relational Database**.

Work breakdown item:

1.7. Quality of service constraints

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- **Quality of Service Constraints**
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

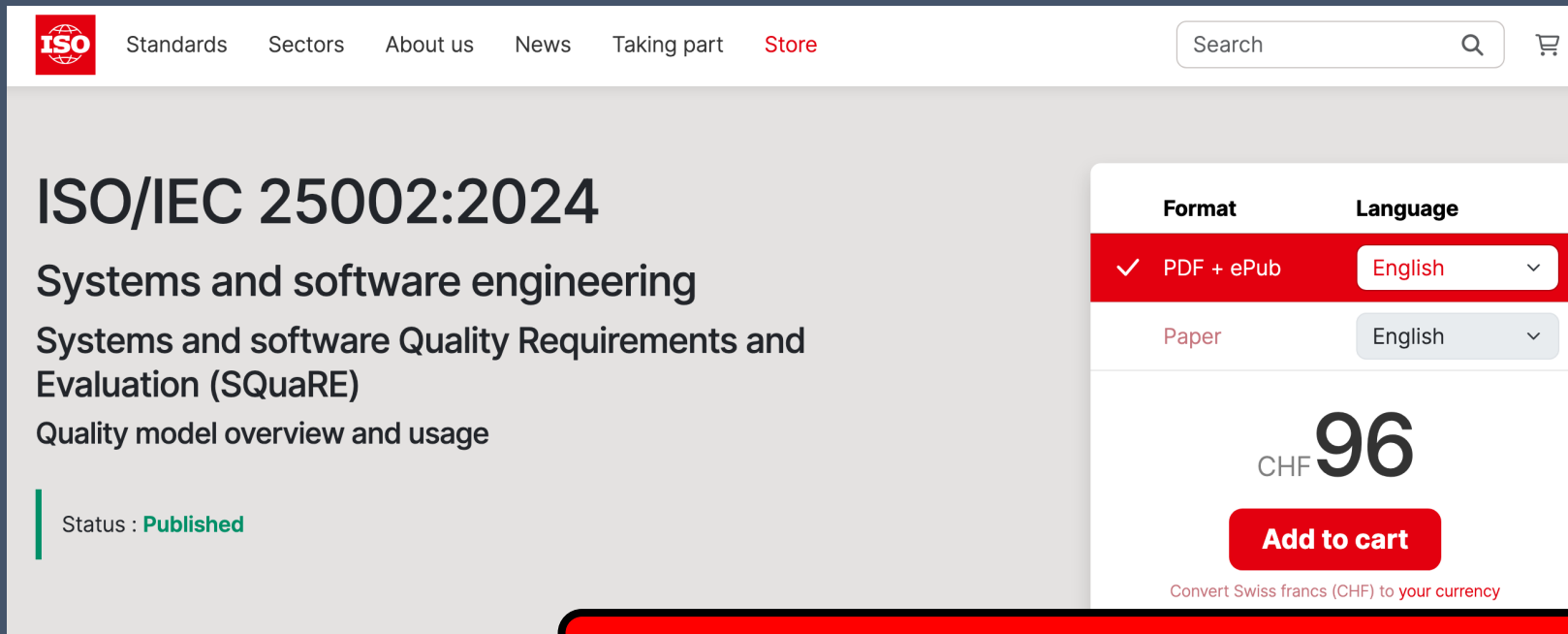
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
1.7. Quality of service constraints

References used for this work breakdown item



The screenshot shows the ISO website's product page for ISO/IEC 25002:2024. The page includes the ISO logo, navigation links (Standards, Sectors, About us, News, Taking part, Store), a search bar, and a shopping cart icon. The main content area displays the standard title 'ISO/IEC 25002:2024', its subtitle 'Systems and software engineering Systems and software Quality Requirements and Evaluation (SQuaRE)', and a description 'Quality model overview and usage'. The status is marked as 'Published'. A purchase summary box on the right shows the format 'PDF + ePub' selected, the language 'English', and a price of 'CHF 96'. An 'Add to cart' button is present, along with a link to convert Swiss francs to the user's currency.

ISO Standards Sectors About us News Taking part Store

Search

ISO/IEC 25002:2024

Systems and software engineering
Systems and software Quality Requirements and Evaluation (SQuaRE)
Quality model overview and usage

Status : **Published**

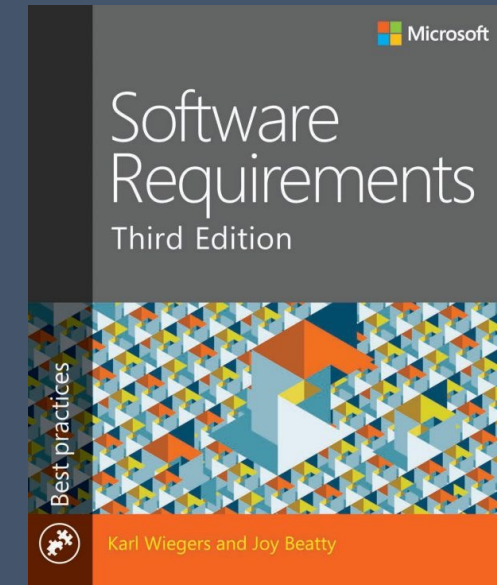
Format	Language
✓ PDF + ePub	English
Paper	English

CHF **96**

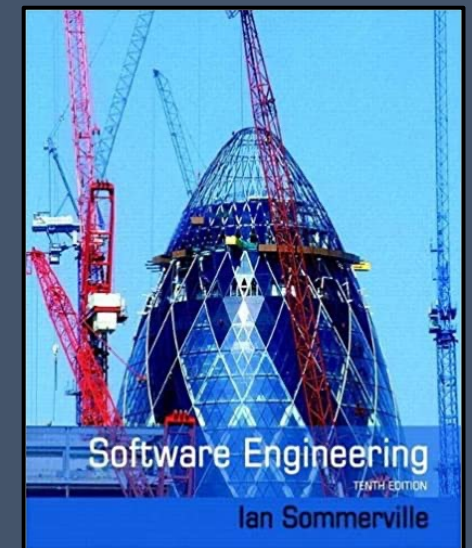
Add to cart

Convert Swiss francs (CHF) to your currency

<https://tinyurl.com/mry4dtaj>

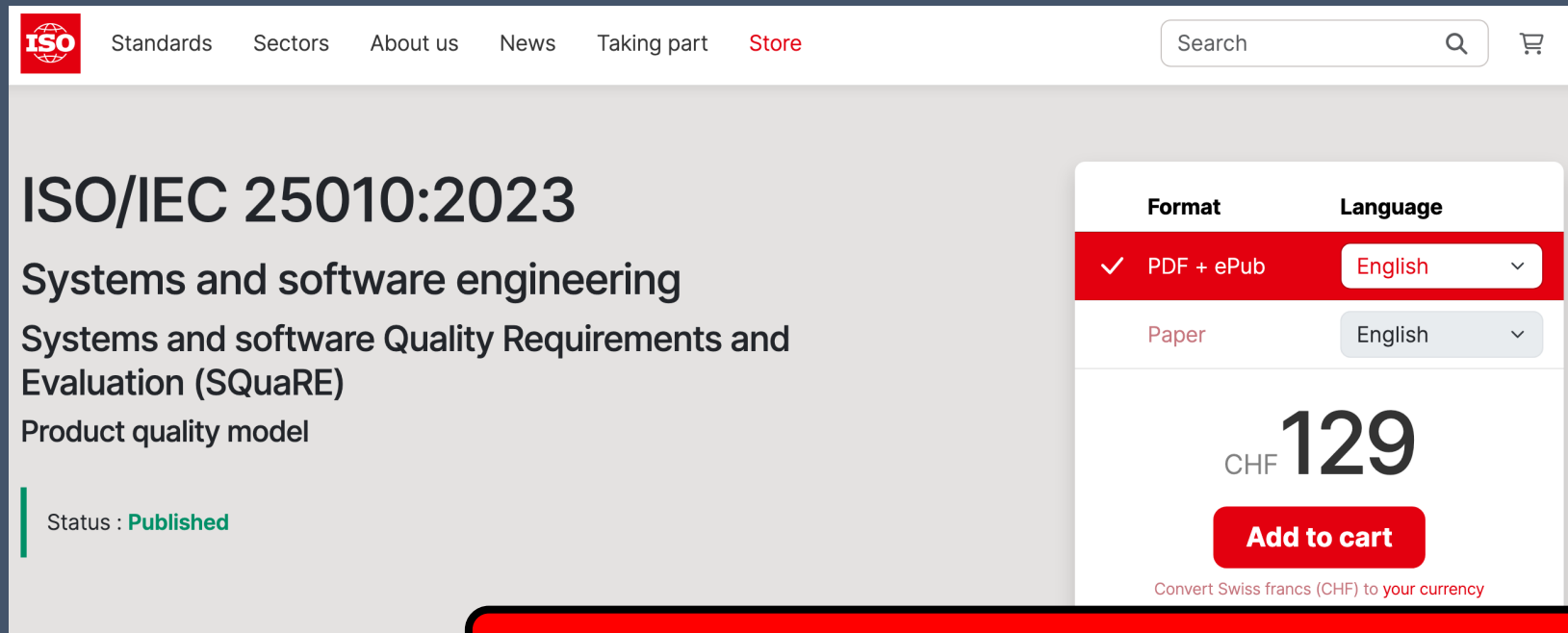


Chapter 1, pages 7-12



Chapter 13

References used for this work breakdown item



The screenshot shows the ISO website's product page for ISO/IEC 25010:2023. The top navigation bar includes links for Standards, Sectors, About us, News, Taking part, and Store. The main heading is 'ISO/IEC 25010:2023' followed by 'Systems and software engineering' and 'Systems and software Quality Requirements and Evaluation (SQuaRE)'. Below this, it says 'Product quality model'. A status bar indicates 'Status : Published'. On the right, there are dropdown menus for 'Format' (with 'PDF + ePub' selected) and 'Language' (with 'English' selected). The price is listed as 'CHF 129' with an 'Add to cart' button. A small note at the bottom says 'Convert Swiss francs (CHF) to your currency'.

ISO Standards Sectors About us News Taking part Store

Search

ISO/IEC 25010:2023

Systems and software engineering

Systems and software Quality Requirements and Evaluation (SQuaRE)

Product quality model

Status : Published

Format Language

✓ PDF + ePub English

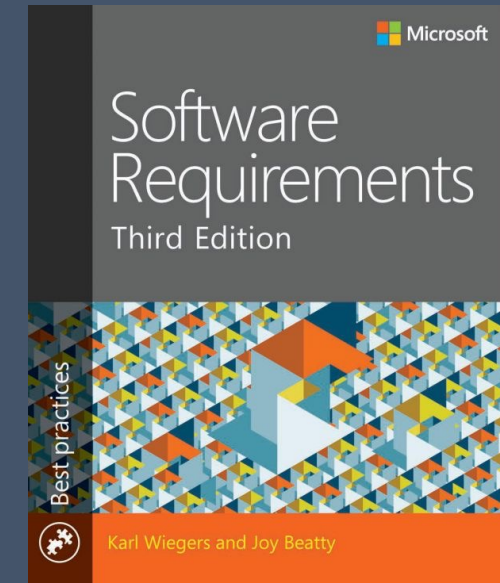
Paper English

CHF 129

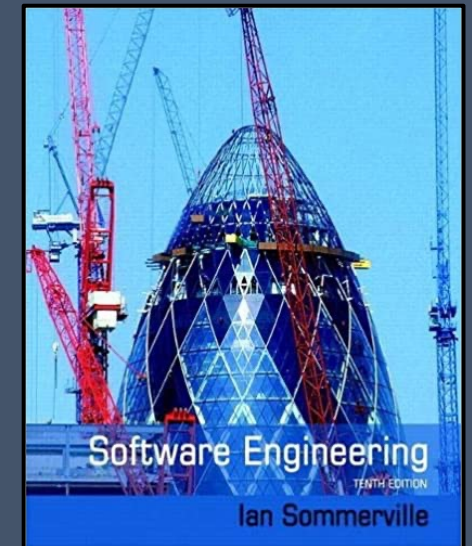
Add to cart

Convert Swiss francs (CHF) to your currency

<https://tinyurl.com/4js2wuk4>

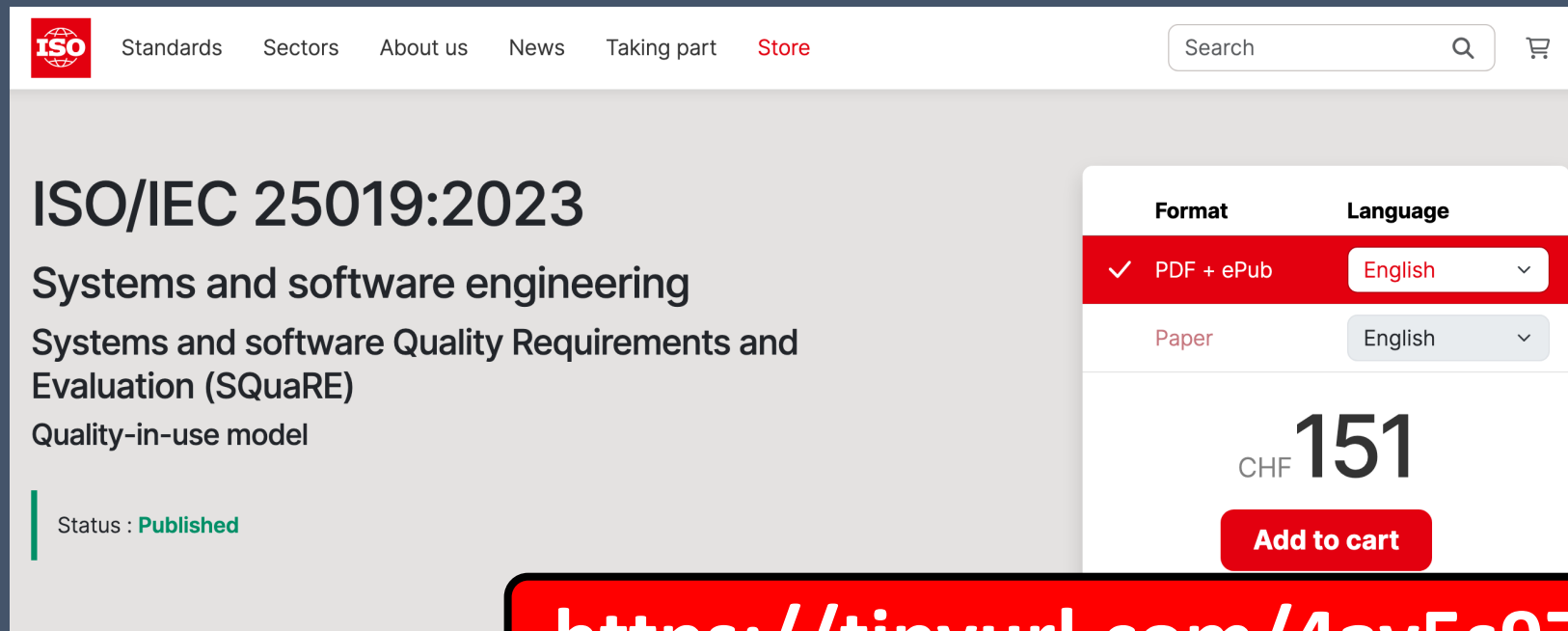


Chapter 1, pages 7-12



Chapter 13

References used for this work breakdown item



ISO/IEC 25019:2023

Systems and software engineering

Systems and software Quality Requirements and Evaluation (SQuaRE)

Quality-in-use model

Status : **Published**

Format **Language**

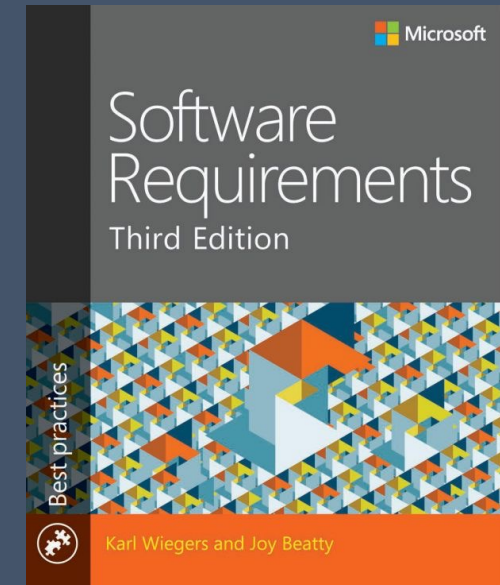
✓ PDF + ePub English

Paper English

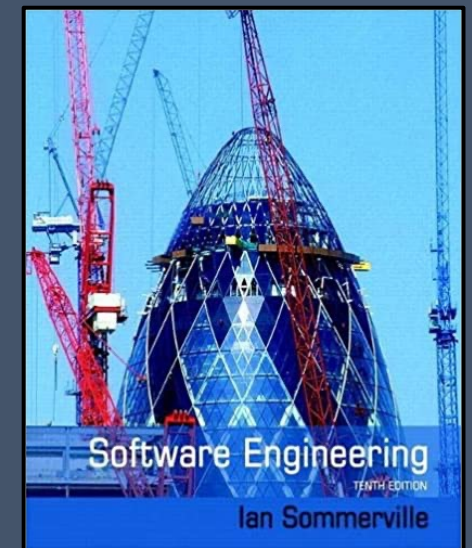
CHF **151**

Add to cart

<https://tinyurl.com/4ay5s97j>



Chapter 1, pages 7-12



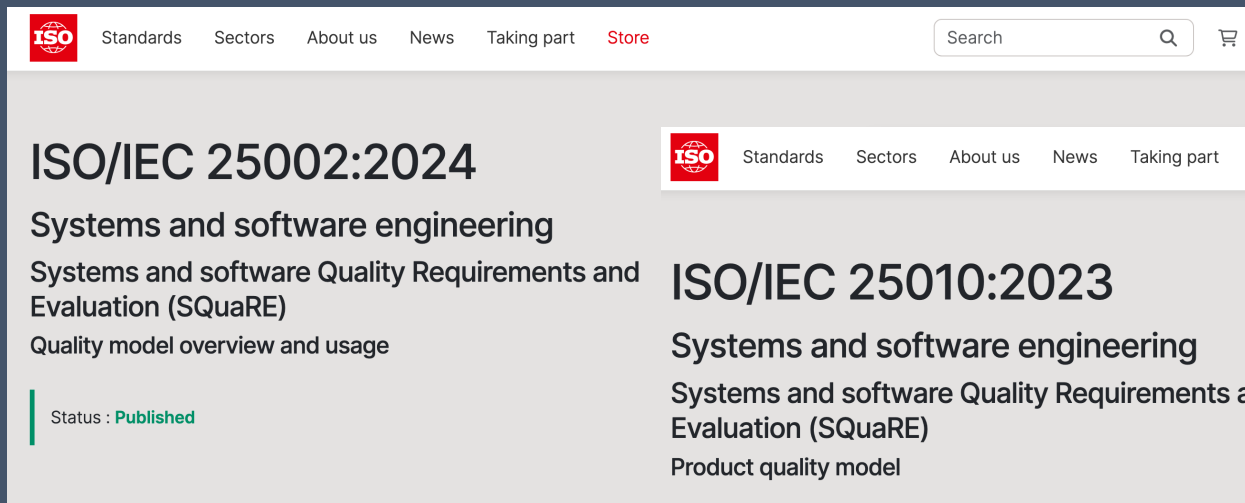
Chapter 13

Quality of service constraints

- Specify **acceptable performance levels** an **automated solution** must **exhibit**.
- **Examples:**
 - ✓ Response time
 - ✓ Throughput
 - ✓ Accuracy
 - ✓ Reliability
 - ✓ Scalability

Quality of service constraints

- ISO/IEC 25002, 25010, and 25019 contain a large list of the kinds of service qualities that can be relevant for software.



ISO/IEC 25002:2024

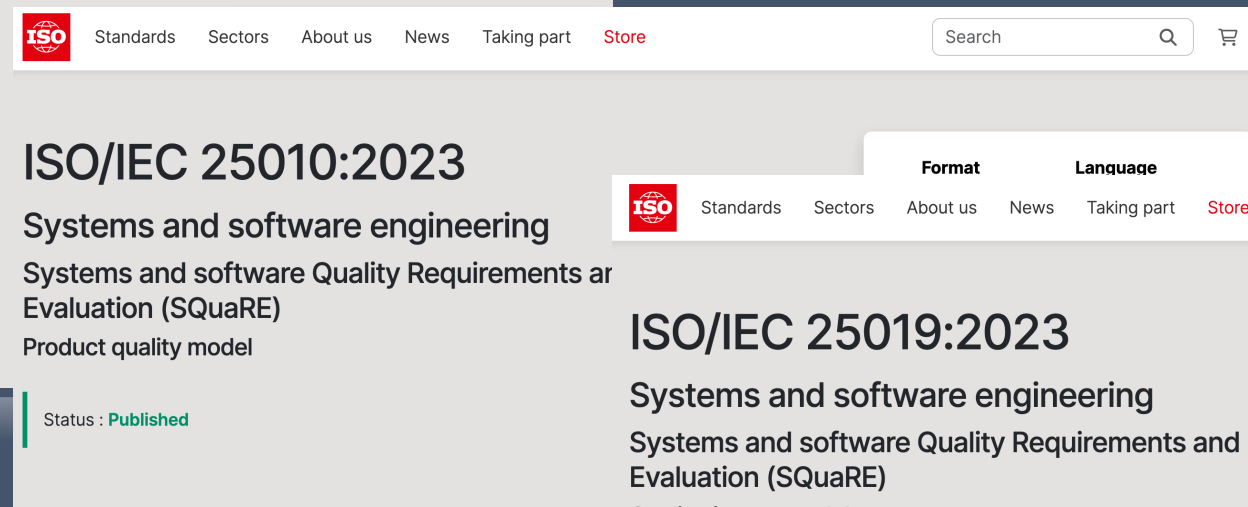
Systems and software engineering

Systems and software Quality Requirements and Evaluation (SQuaRE)

Quality model overview and usage

Status : **Published**

This screenshot shows the top portion of the ISO/IEC 25002:2024 product page. It includes the ISO logo, navigation links (Standards, Sectors, About us, News, Taking part, Store), a search bar, and a shopping cart icon. The main content area displays the title 'ISO/IEC 25002:2024', the domain 'Systems and software engineering', the full title 'Systems and software Quality Requirements and Evaluation (SQuaRE)', and a subtitle 'Quality model overview and usage'. The status is indicated as 'Published'.



ISO/IEC 25010:2023

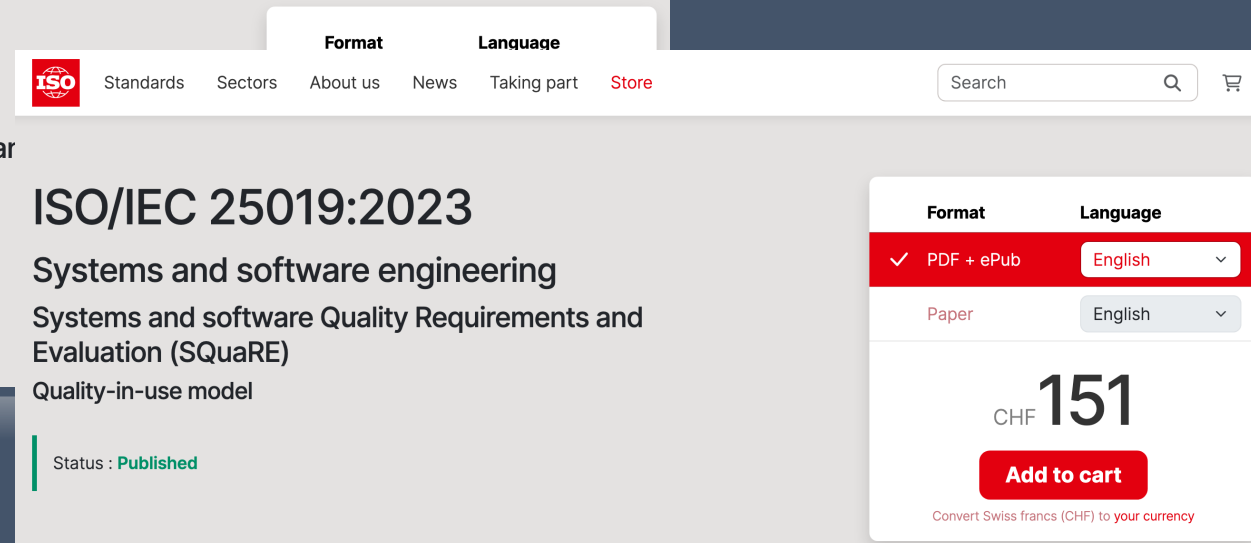
Systems and software engineering

Systems and software Quality Requirements and Evaluation (SQuaRE)

Product quality model

Status : **Published**

This screenshot shows the top portion of the ISO/IEC 25010:2023 product page. It includes the ISO logo, navigation links, a search bar, and a shopping cart icon. The main content area displays the title 'ISO/IEC 25010:2023', the domain 'Systems and software engineering', the full title 'Systems and software Quality Requirements and Evaluation (SQuaRE)', and a subtitle 'Product quality model'. The status is indicated as 'Published'.



ISO/IEC 25019:2023

Systems and software engineering

Systems and software Quality Requirements and Evaluation (SQuaRE)

Quality-in-use model

Status : **Published**

Format

✓ PDF + ePub

Language

English

Paper

English

CHF **151**

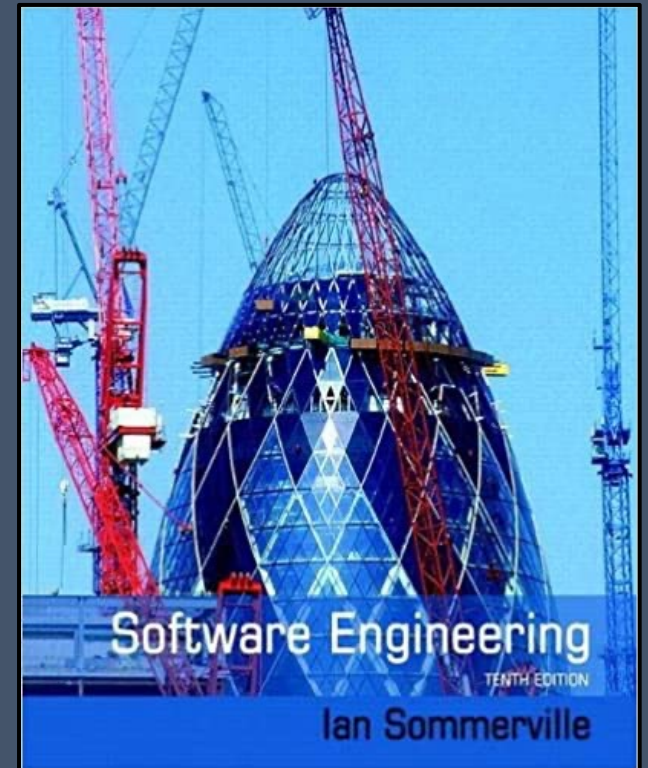
Add to cart

Convert Swiss francs (CHF) to [your currency](#)

This screenshot shows the bottom portion of the ISO/IEC 25019:2023 product page, focusing on purchase options. It includes the ISO logo, navigation links, a search bar, and a shopping cart icon. The main content area displays the title 'ISO/IEC 25019:2023', the domain 'Systems and software engineering', the full title 'Systems and software Quality Requirements and Evaluation (SQuaRE)', and a subtitle 'Quality-in-use model'. The status is indicated as 'Published'. Below this, there are two tabs: 'Format' and 'Language'. The 'Format' tab is selected, showing 'PDF + ePub' as the chosen option. The 'Language' tab is also selected, showing 'English' as the chosen option. The price is displayed as 'CHF 151'. There is a red 'Add to cart' button. At the bottom, there is a link to 'Convert Swiss francs (CHF) to your currency'.

Quality of service constraints

- **Security** is also a particularly **important topic** where **requirements tend to be overlooked**.
- See the **Security KA** for **details** on the **kinds of specific security requirements** that should be **considered**.
- See also I. Sommerville, **Software Engineering**, 10th ed, 2018.



Work breakdown item:

1.8. Why categorize requirements this way?

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
 - Why Categorize Requirements This Way? 1.8
 - System Requirements and Software Requirements
 - Derived Requirements
 - Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

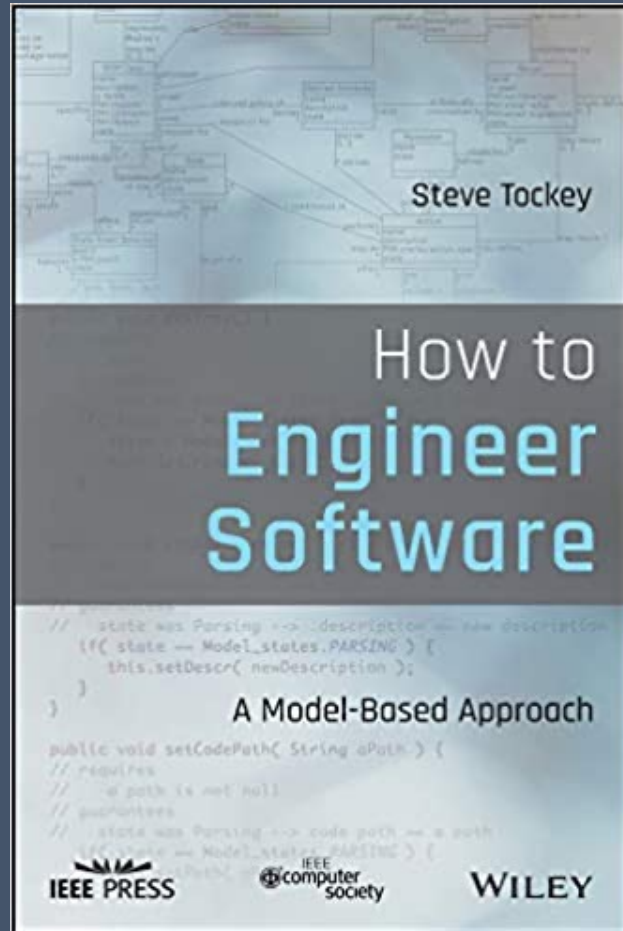
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

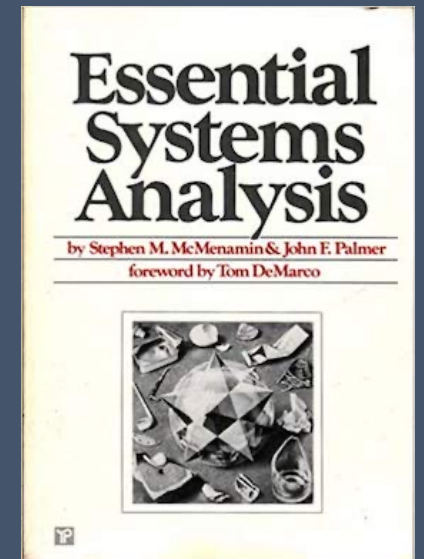
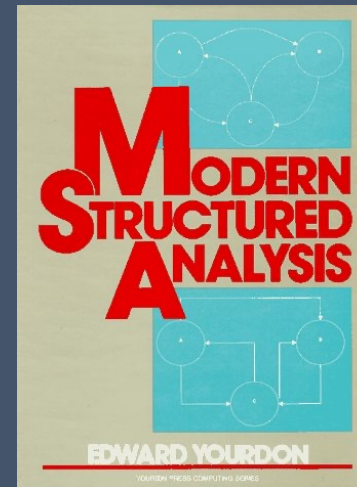
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
1.8. Why categorize requirements this way?**

References used for this work breakdown item



Chapter 4



Chapters 1-4

Why is useful categorize requirements?

- Requirements in one category tend to come from different sources than other categories.
- Elicitation techniques often vary by source.
- Analysis techniques vary by category.

Why is useful categorize requirements?

- Specification techniques vary by category
- Validation authorities vary by category.
- The different categories affect the resulting software in different ways.

Benefits of categorize requirements

- **Complexity** can be better managed because different areas can be addressed separately.
- **Software engineers** can deal with **policy** and **process complexities** without worrying about **automation technology** issues at the same time
 - ✓ And vice versa.
- One **large problem** becomes **two smaller ones**.
 - ✓ **Divide and conquer complexity** management.

Benefits of categorize requirements

- **Distinct areas of expertise can be isolated.**
 - ✓ **Stakeholders, not software engineers, are the experts in the policies and processes to be automated.**
 - ✓ **Software engineers, not stakeholders, are the technology experts.**
 - ✓ **Business usually don't understand — or even care about — technology issues.**
 - ✓ **The relevant requirements reviewer can focus on just the subset of requirements relevant to them.**

The perfect technology filter

- Helps separate **functional** from **nonfunctional requirements**.
- **Functional requirements** are those that would still need to be stated even if a **supercomputer** existed on which to **construct the software**.
 - ✓ With infinite speed, unlimited memory, zero cost, no failures, etc.
- All other **software product requirements** provided by our **hypothetical supercomputer** are **constraints on automation technologies** and are therefore **nonfunctional**.

Work breakdown item:

1.9. System requirements and software requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
 - Why Categorize Requirements This Way?
 - **System Requirements and Software Requirements 1.9**
 - Derived Requirements
 - Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis

Requirements Specification

- Unstructured Natural Language Requirements

Requirements Validation

- Requirements Reviews
- Simulation and Prototyping

Requirements Management Activities

- Requirements Scrubbing

Practical Considerations

- Iterative Nature of the Requirements Process

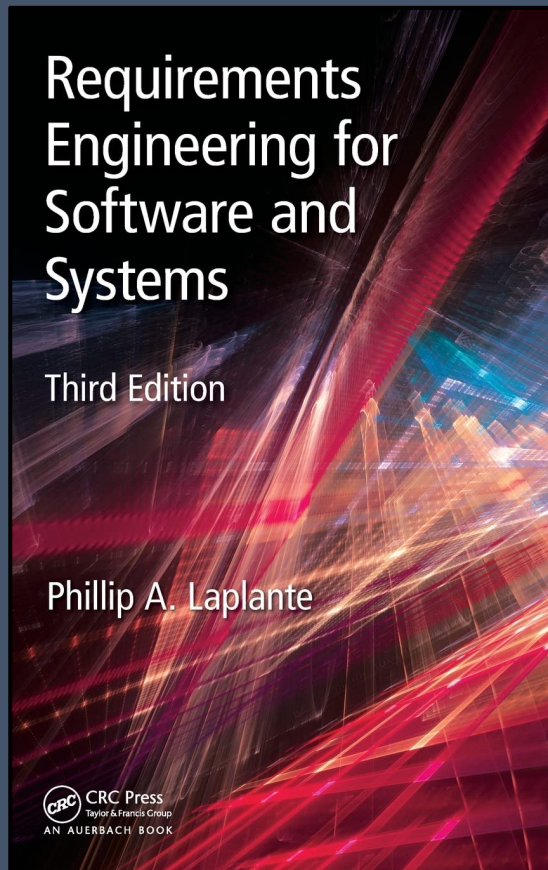
Software Requirements Tools

- Requirements Management Tools

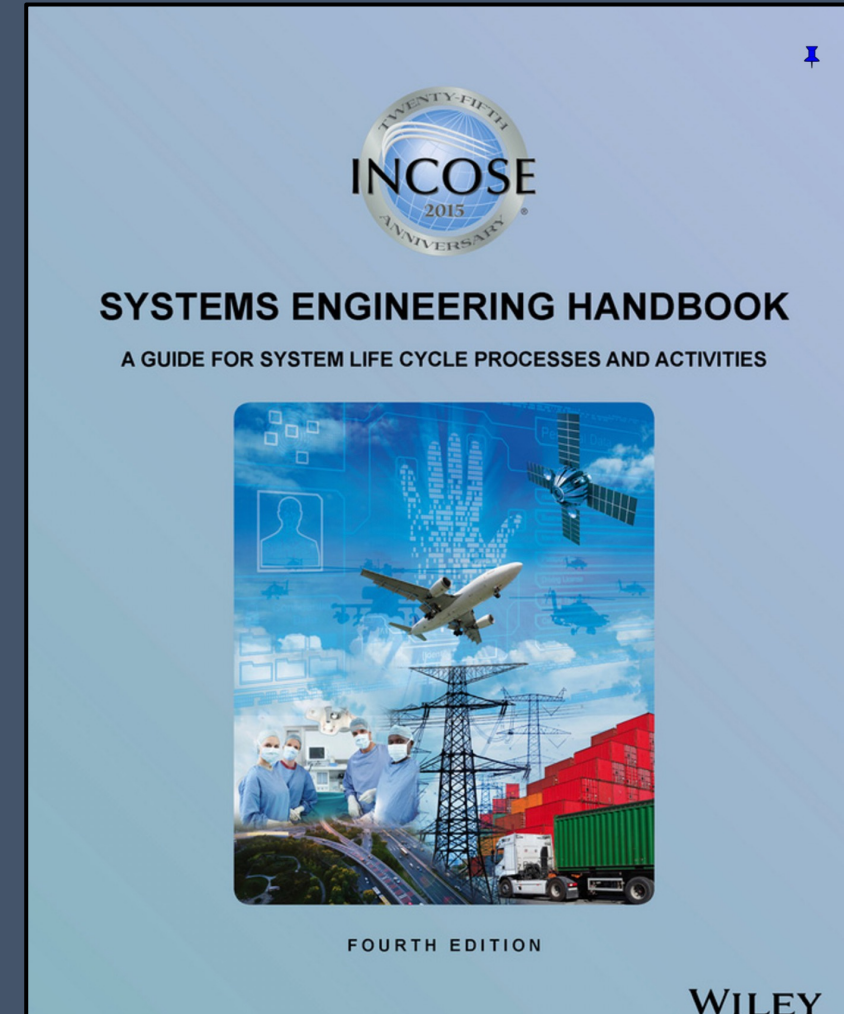
**Work breakdown item:
1.9. System requirements and software requirements**

- Model-Based Requirements Specification
- Additional Attributes of Requirements
- Incremental and Comprehensive Requirements Specification

References used for this work breakdown item



Chapter 1

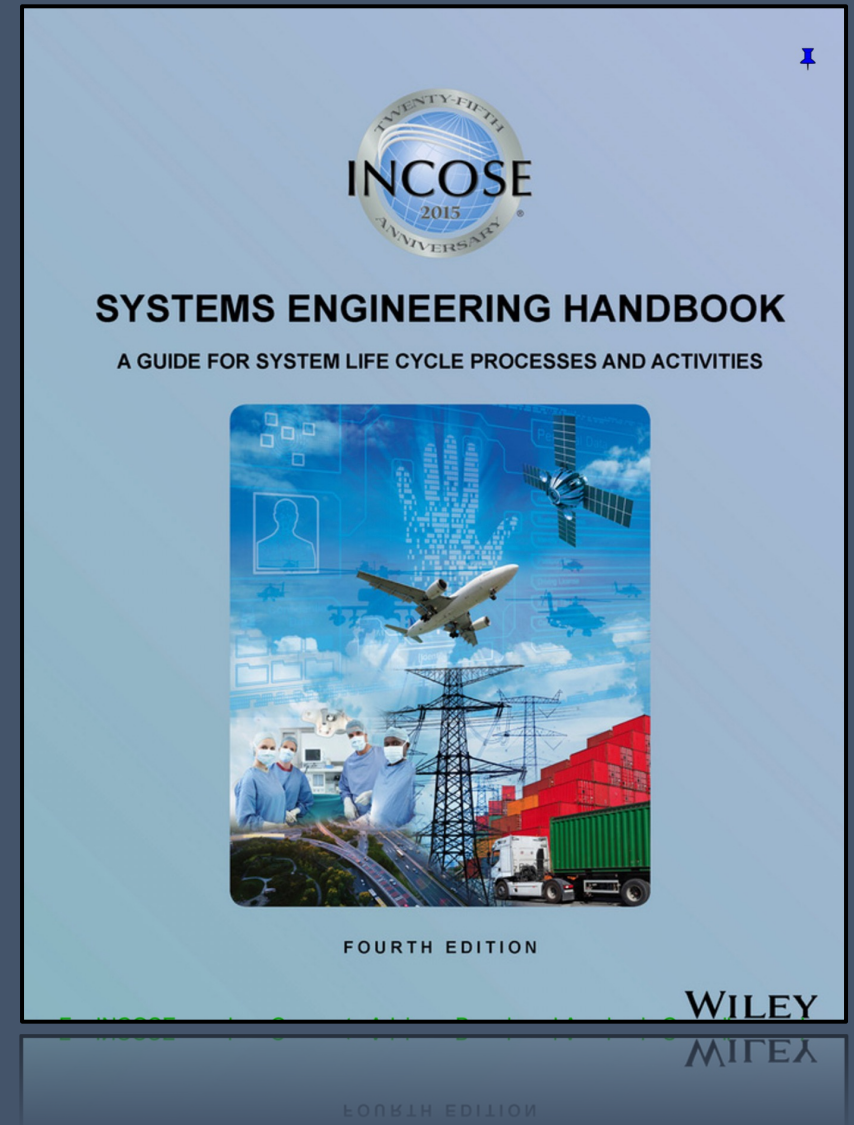


Definition of system

- The International Council on Systems Engineering (INCOSE) defines a **system** as:

“A combination of interacting elements organized to achieve one or more stated purposes”.

- System includes hardware, software, firmware, people, information, techniques, facilities, services, and other support elements”



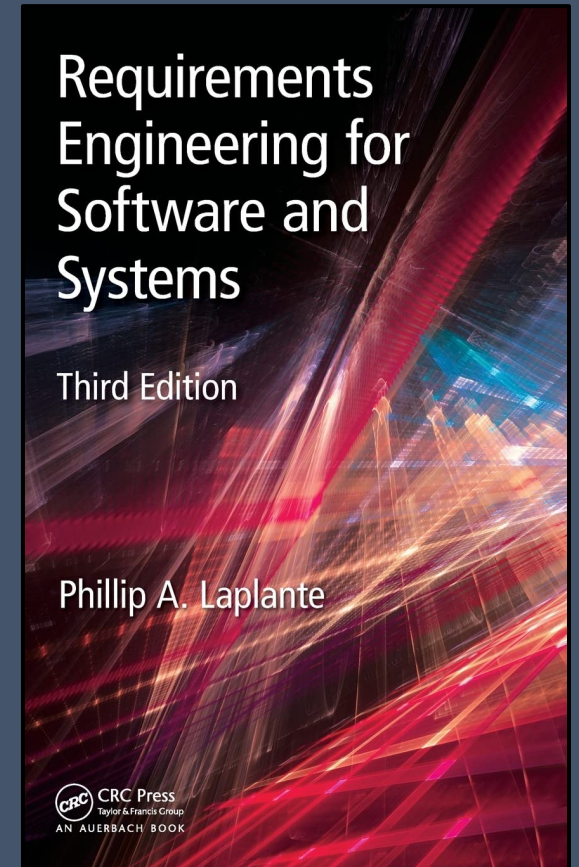
System and software requirements

- In some cases, it is either **useful** or **mandatory** to **distinguish system requirements** from **software requirements**.
- **System requirements apply to larger systems**
 - ✓ For example, an **autonomous vehicle**.
- **Software requirements apply only** to an **element of software** in that **larger system**.

System and software requirements

“Some software requirements may be derived from system requirements”.

Laplante, Phillip A. **Requirements engineering for software and systems**, 2017 (Chapter 1)



Work breakdown item:

1.10. Derived requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- **Derived Requirements 1.10**
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements

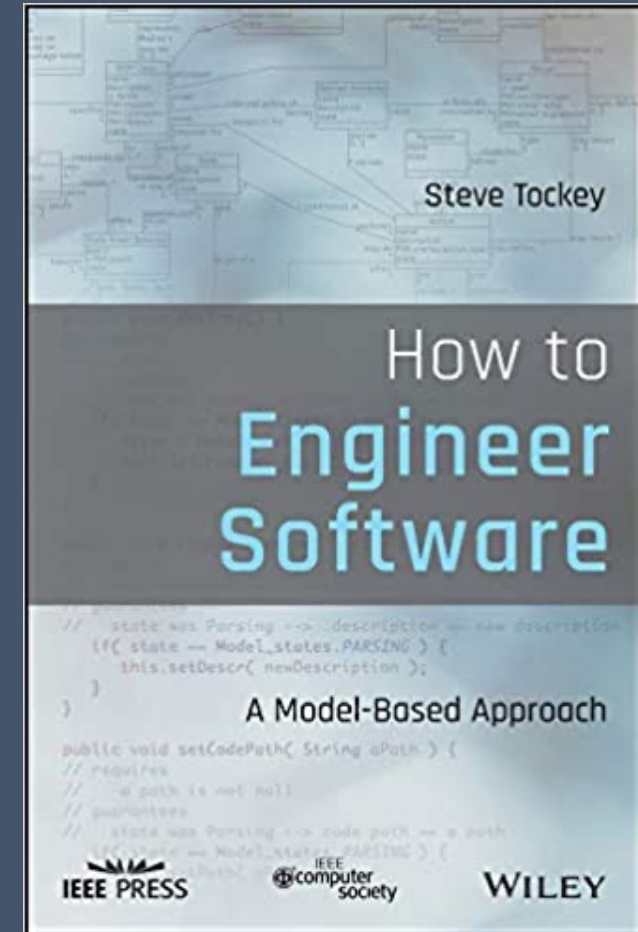
Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
1.10. Derived requirements**

→ Incremental and Comprehensive Requirements Specification

References used for this work breakdown item



Chapter 4

Derived requirements

- The aerospace industry has long used the term **derived requirement**
 - ✓ A requirement that was **not made** by a stakeholder external to the **overall project** but that was **imposed** inside the **larger development team**.

Work breakdown item:

1.11. Software requirements activities

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

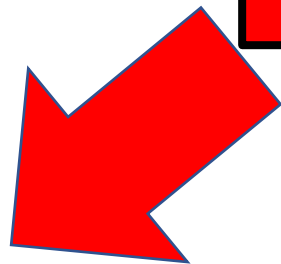
Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements

Software Requirements Tools

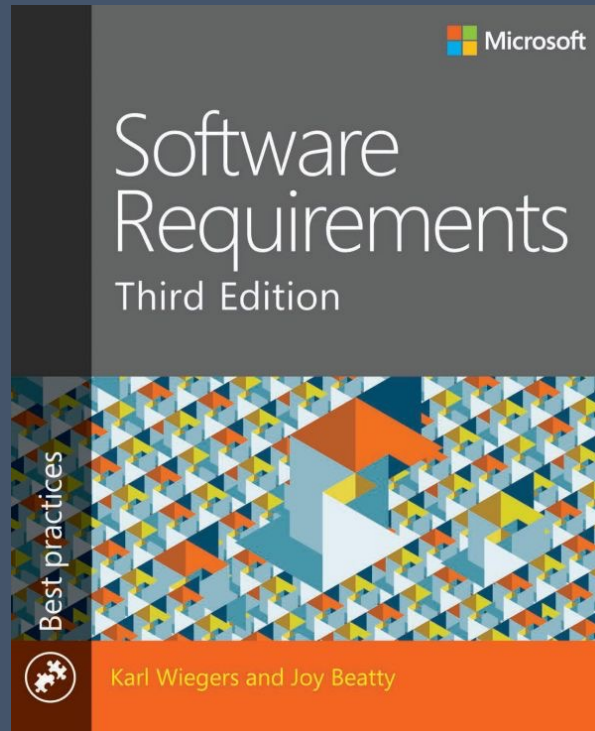
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
1.11. Software requirements activities

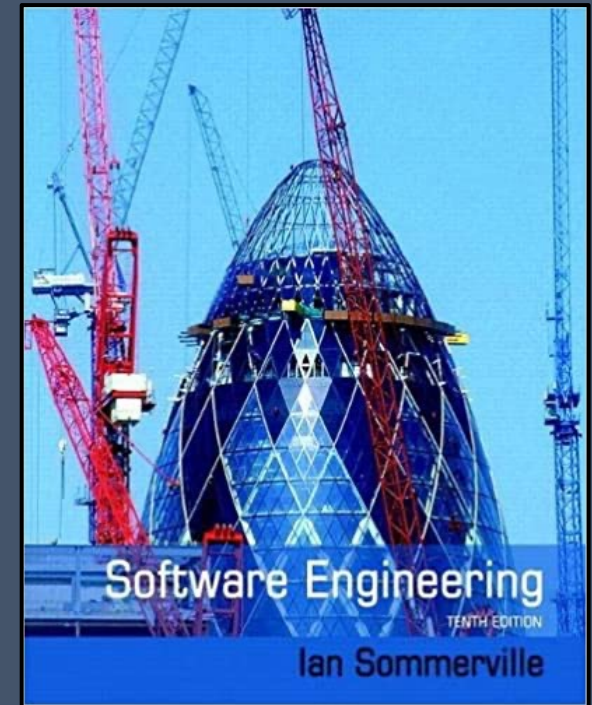


1.11

References used for this work breakdown item

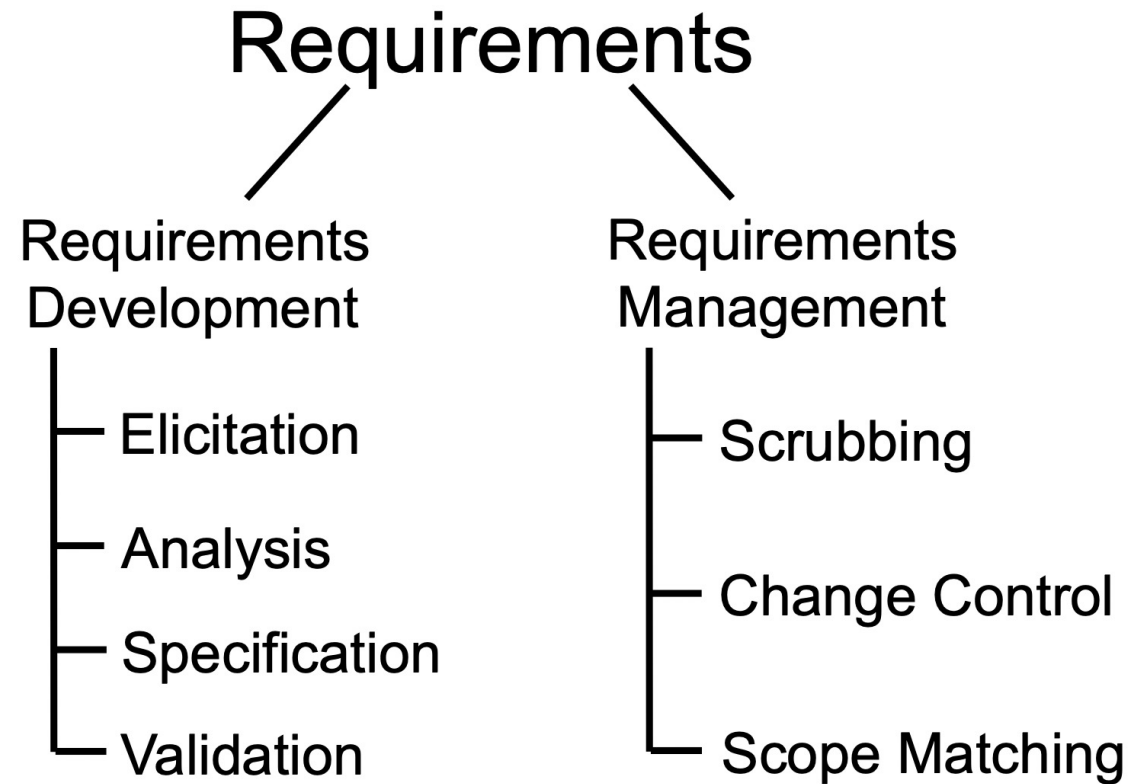


Chapter 1, pages 15-18



Section 4.2

Software requirements activities



Requirements development vs. management

- Requirements development:

“Reaching an agreement on what software is to be constructed.”

- Requirements management:

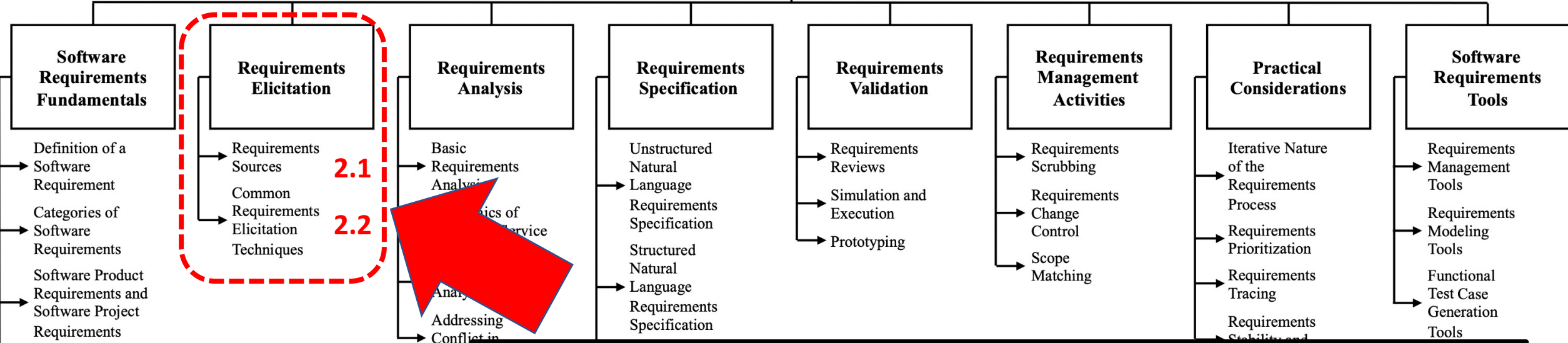
“Maintaining that agreement over time.”

Work Breakdown item:

2. Requirements elicitation

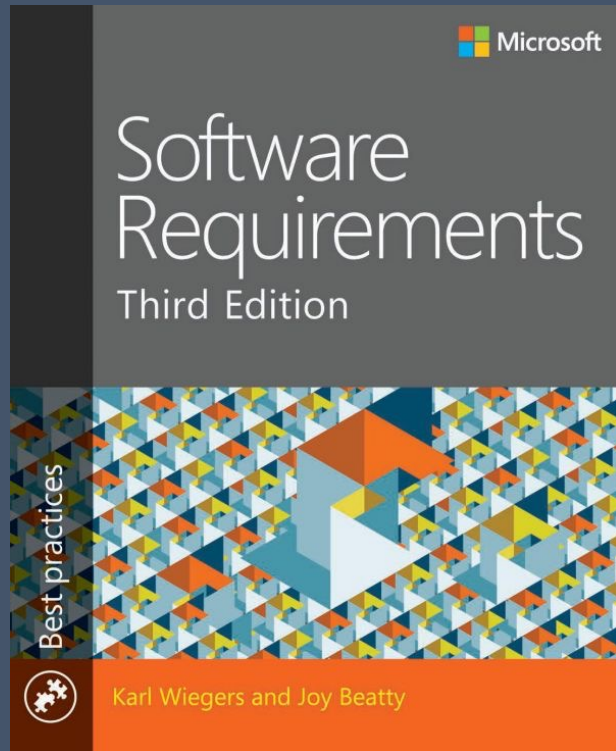
2

Software Requirements

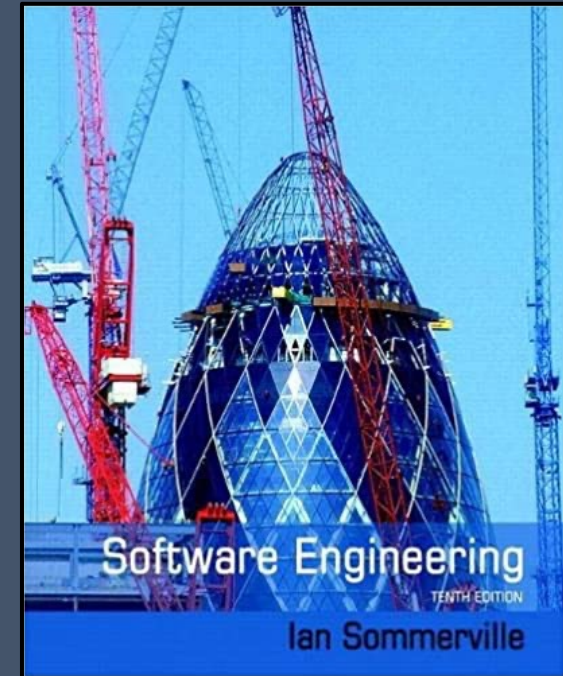


Work breakdown item:
2. Requirements elicitation

References used for this work breakdown item



Chapters 6 and 7



Section 4.3

Introduction

The goal of requirements elicitation

- Surface candidate requirements.
- Requirements elicitation is also called:
 - ✓ Requirements capture
 - ✓ Requirements discovery
 - ✓ Requirements acquisition.

Good elicitation minimizes incompleteness

- One problem in requirements work on real-world software projects is incompleteness.
 - ✓ This can be the result of inadequate elicitation.
- Although there is no guarantee that a set of requirements is complete, well-executed elicitation helps minimize incompleteness.

Work breakdown item:

2.1. Requirements sources

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Requirements Analysis
- Quality of Service Requirements
- Analyzing Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Incremental and Comprehensive Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

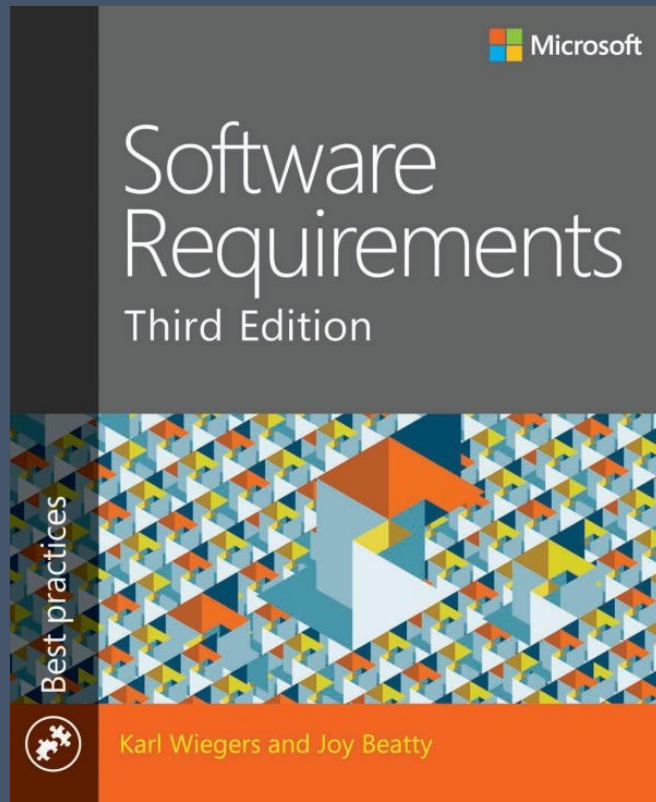
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Management

Software Requirements Tools

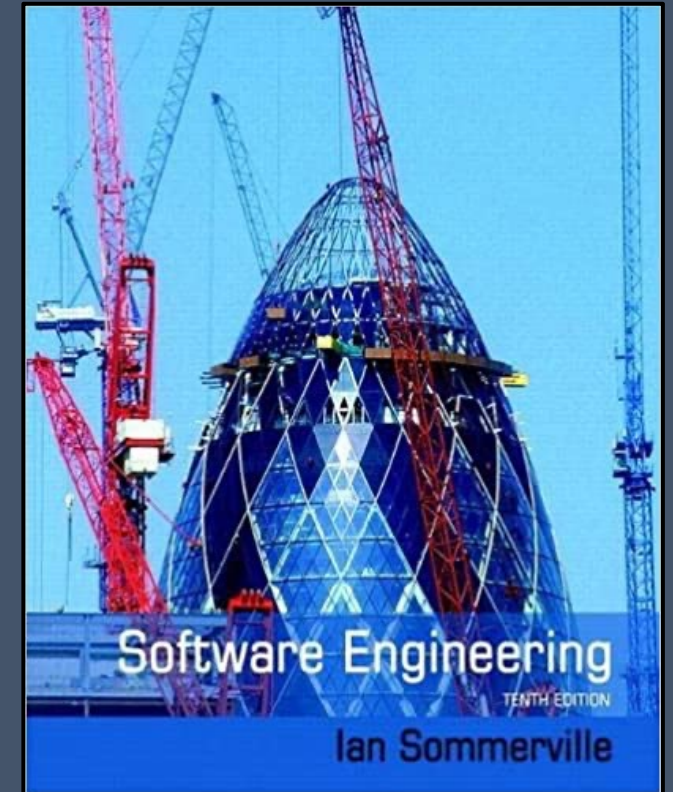
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
2.1. Requirements sources

References used for this work breakdown item



Chapter 6



Section 4.3

Definition of stakeholder

- A stakeholder is any person, group or organization that:
 - ✓ ... is actively involved in the project.
 - ✓ ... is affected by the project's outcome
 - ✓ ...can influence the project's outcome.
- A stakeholder is one possible source of requirements.

Typical stakeholders for software projects

- Clients
- Customers
- Users
- Subject Matter Experts (SMEs)
- Operation staff
- First-level product support staff
- Relevant professional bodies
- Regulatory agencies
- Special interest groups
- People who can be negatively affected if the project is successful
- Developers
- ...
- ...

Work breakdown item:

2.2. Common requirements elicitation techniques

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
 - Why Categorize Requirements This Way?
 - System Requirements and Software Requirements
 - Derived Requirements
 - Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

2.2

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Modeling
- Assessing Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

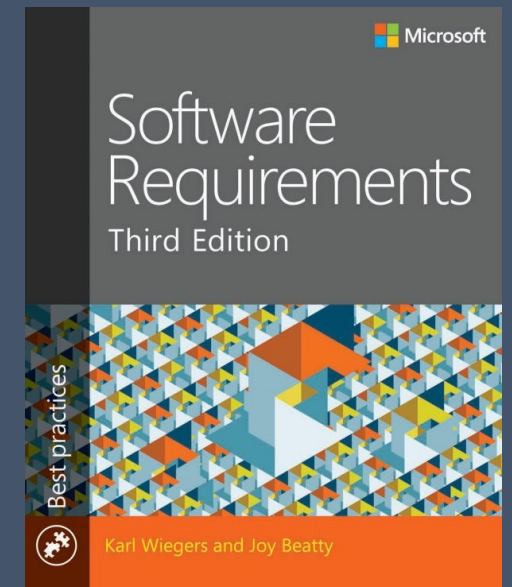
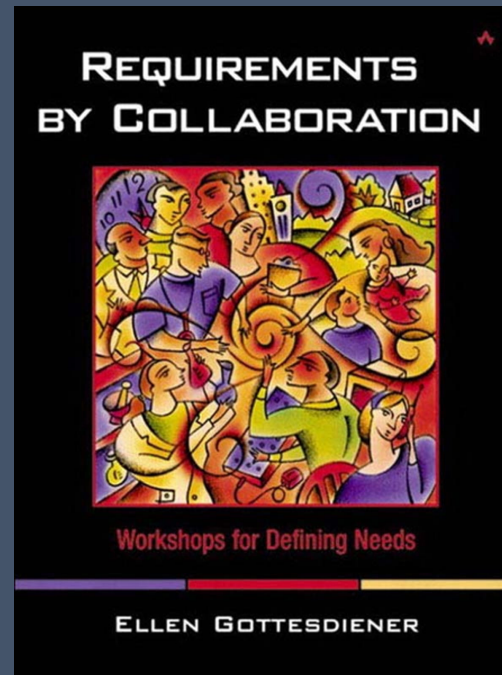
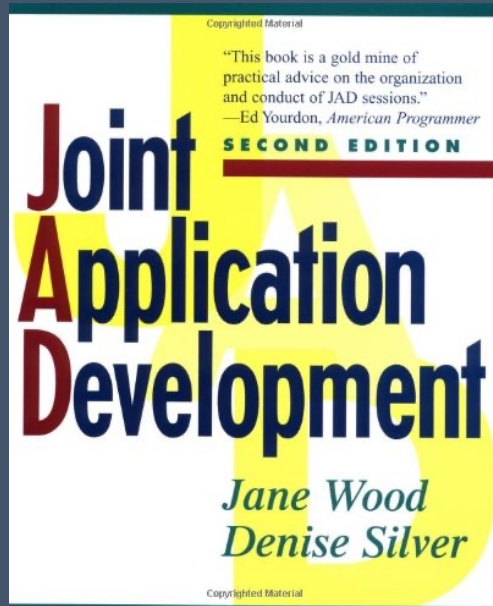
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

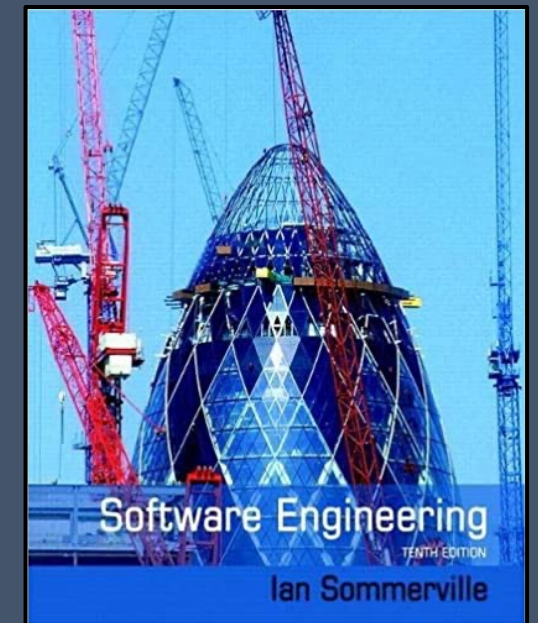
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
2.2. Common Requirements Elicitation Techniques

References used for this work breakdown item



Chapter 7



Section 4.3

Requirements elicitation techniques

- Interviews, meetings, possibly including brainstorming.
- Joint Application Development (JAD), Joint Requirements Planning (JRP) and other facilitated workshops.
- Protocol analysis.
- Focus groups.

Requirements elicitation techniques

- Questionnaires and market surveys.
- **Exploratory prototyping**
 - ✓ Including **low-fidelity** and **high-fidelity** user interface **prototyping**.
- **User story mapping.**

Requirements elicited from other sources

- Previous versions of the system.
- Defect tracking database for previous versions of the system.
- Systems that interface with the system under development.
- Competitive benchmarking.

Requirements elicited from other sources

- Literature search.
- Quality Function Deployment (QFD)'s House of Quality
- Observation
 - ✓ Where the **software engineer studies the work** and the **environment** where the **work is being done**.



Requirements elicited from other sources

- **Apprenticing**
 - ✓ Where the **software engineer** learns by **doing the work**.
- **Usage scenario descriptions.**
- **Decomposition**
 - ✓ E.g., **capabilities** into **epics** into **features** into **stories**.
- **Task analysis**

Requirements elicited from other sources

- **Design thinking**
 - ✓ Empathize, define, ideate, prototype, test.
- **ISO/IEC 25002, 25010, and 25019**
- **Security requirements**
 - ✓ More details in Security KA of SWEBOK.
- **Standards and regulations.**

Work Breakdown item:

3. Requirements analysis

Software Requirements

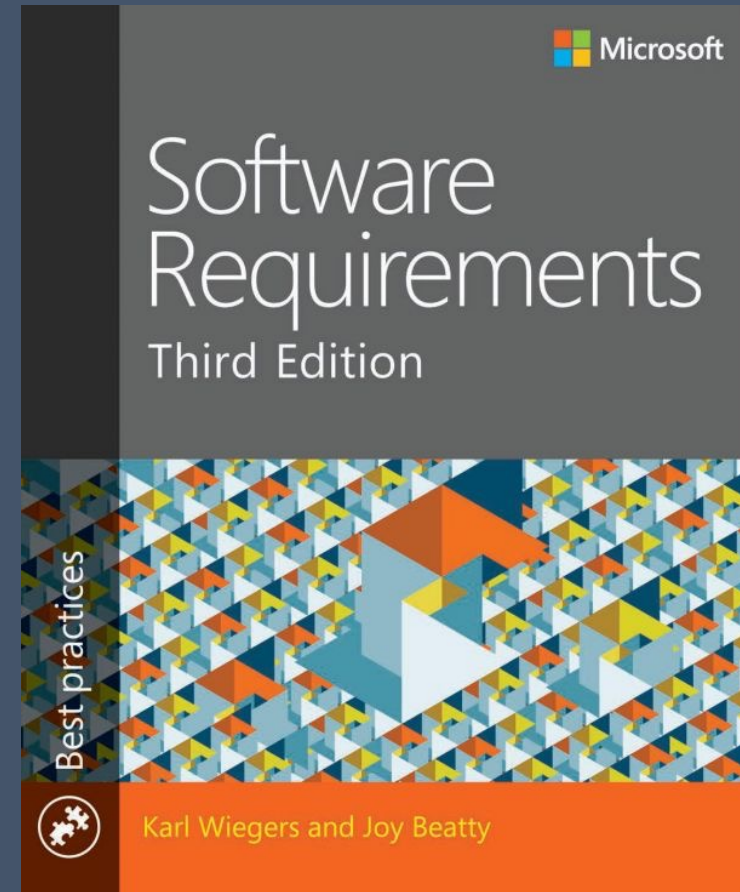
3

Requirements Analysis

- Basic Requirements Analysis 3.1
- Economics of Quality of Service Constraints 3.2
- Formal Analysis 3.3
- Addressing Conflict in Requirements 3.4

Work breakdown item:
3. Requirements analysis

References used for this work breakdown item



Chapters 8 and 9

Introduction

Requirements analysis

- Requirements are unlikely to be elicited in their final form.
- Further investigation is usually needed to reveal the full, true requirements suggested by the originally elicited information.

Work breakdown item:

3.1. Basic Requirements Analysis

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
 - Why Categorize Requirements This Way?
 - System Requirements and Software Requirements
 - Derived Requirements
 - Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis **3.1**
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language
- Structured Natural Language
- Formal Specification
- Acceptance Criteria

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

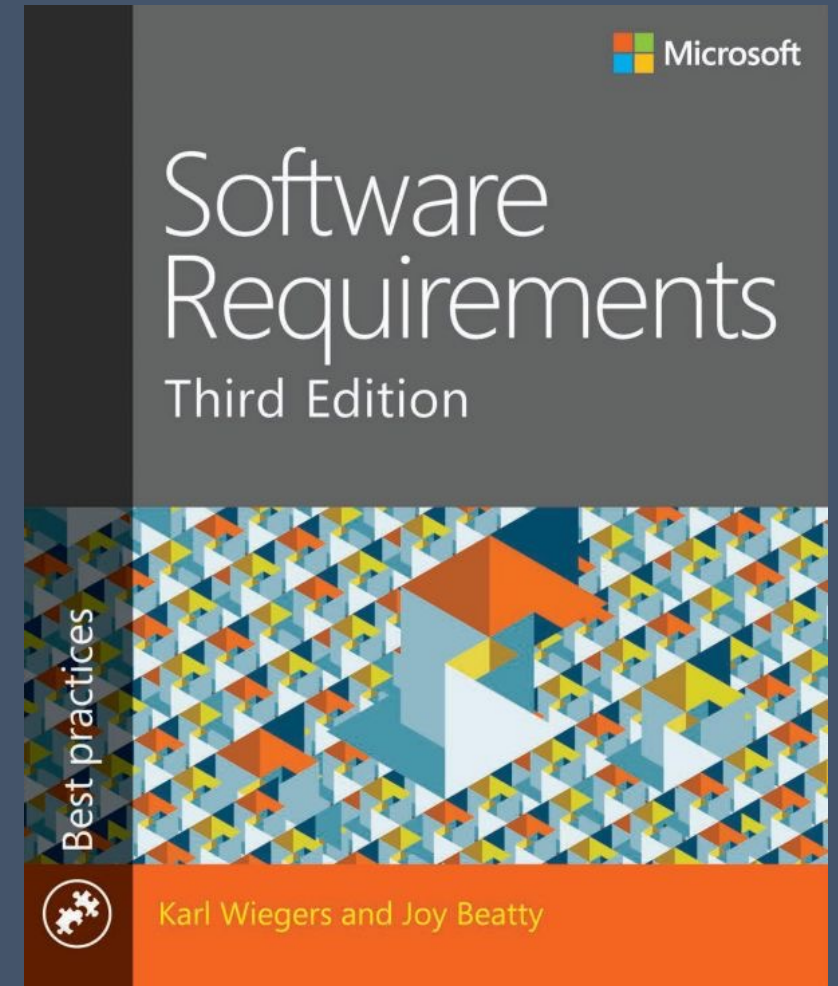
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Change

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
3.1. Basic requirements analysis

References used for this work breakdown item



Chapters 8 and 9

Each requirement should...

Each requirement should...

- Be **unambiguous**.
 - ✓ **Interpretable** in only **one way**.
- Be **testable** (quantified).
 - ✓ **Compliance** or **noncompliance** can be clearly **demonstrated**.
- Be **binding**
 - ✓ Meaning that **clients** are **willing** to **pay** for it and **unwilling not** to **have** it.

Each requirement should... (cont.)

- Represent true, actual stakeholder needs
- Use stakeholder vocabulary.
- Be acceptable to all stakeholders.

The collection of requirement should be...

The collection of requirements should be...

- **Complete**

- ✓ The requirements adequately address boundary conditions, exception conditions and security needs.

- **Internally consistent**

- ✓ No requirement conflicts with any other.

- **Externally consistent**

- ✓ No requirement conflicts with any source material.

- **Feasible**

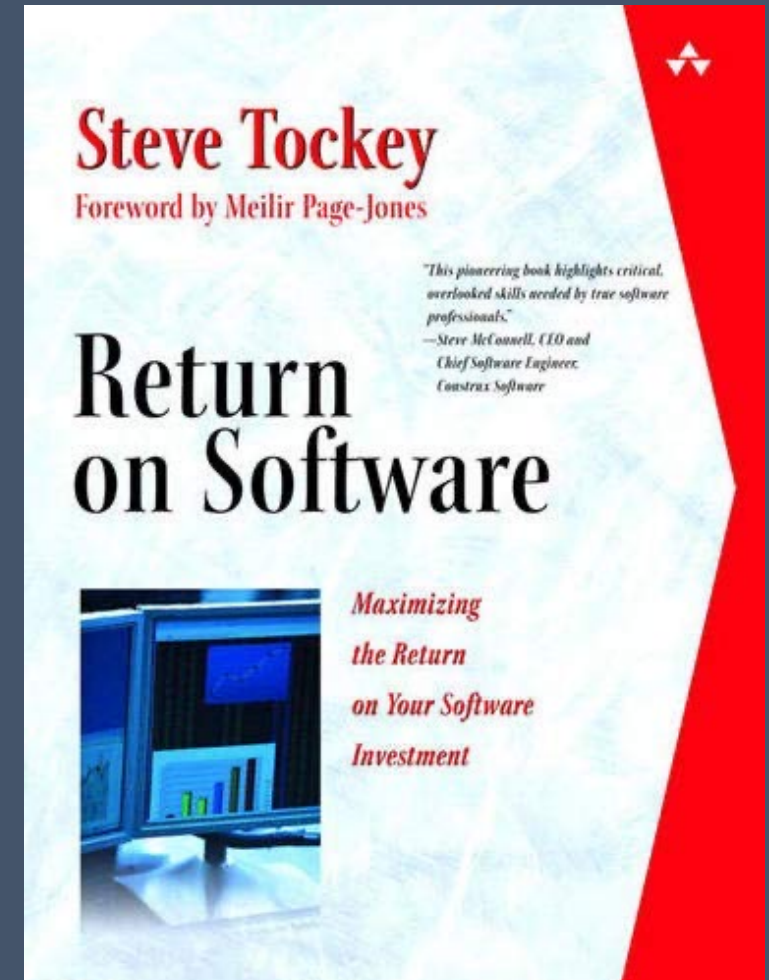
- ✓ A viable, cost-effective solution can be created within cost, schedule, staffing, and other constraints.

Basic requirements analysis

- In some cases, an elicited statement represents a solution to be implemented rather than the true problem to be solved.
 - ✓ This risks implementing a suboptimal solution.

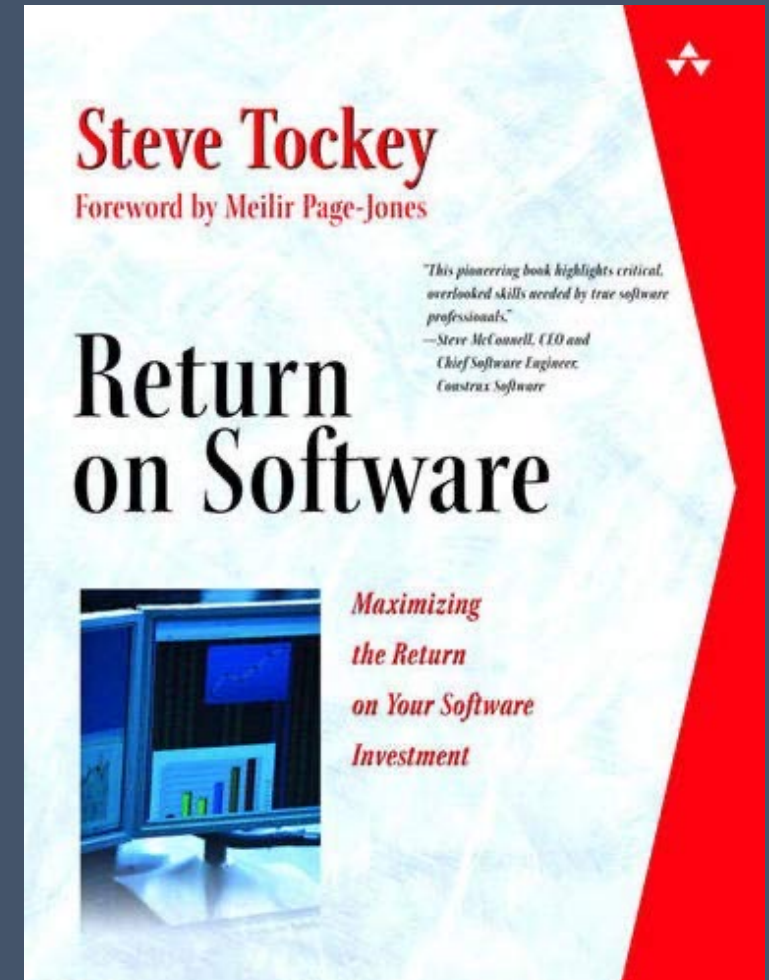
The 5-whys technique

- Involves repeatedly asking
“Why is this the requirement?” to converge
on the true problem.
- Repetition stops when the answer is,
*“If that isn’t done, then the stakeholder’s
problem has not been solved.”*



The 5-whys technique

- Often, the true problem is reached in two or three cycles.



Work breakdown item:

3.2. Economics of Quality of Service Constraints

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints **3.2**
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Requirements Specification
- Accurate Criteria for Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

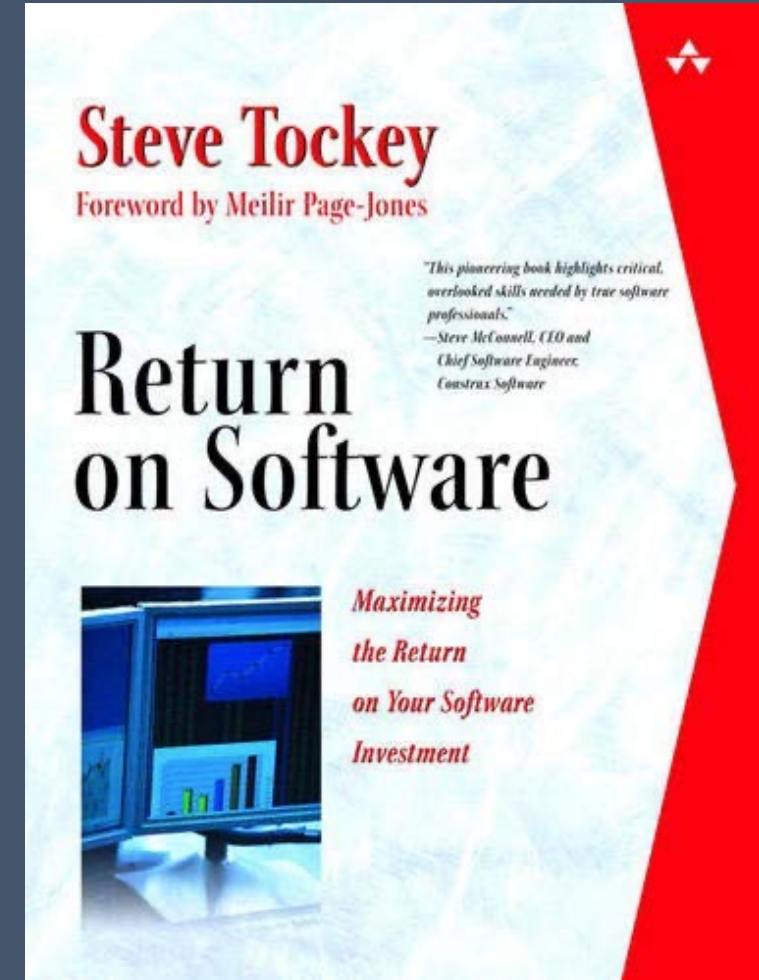
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

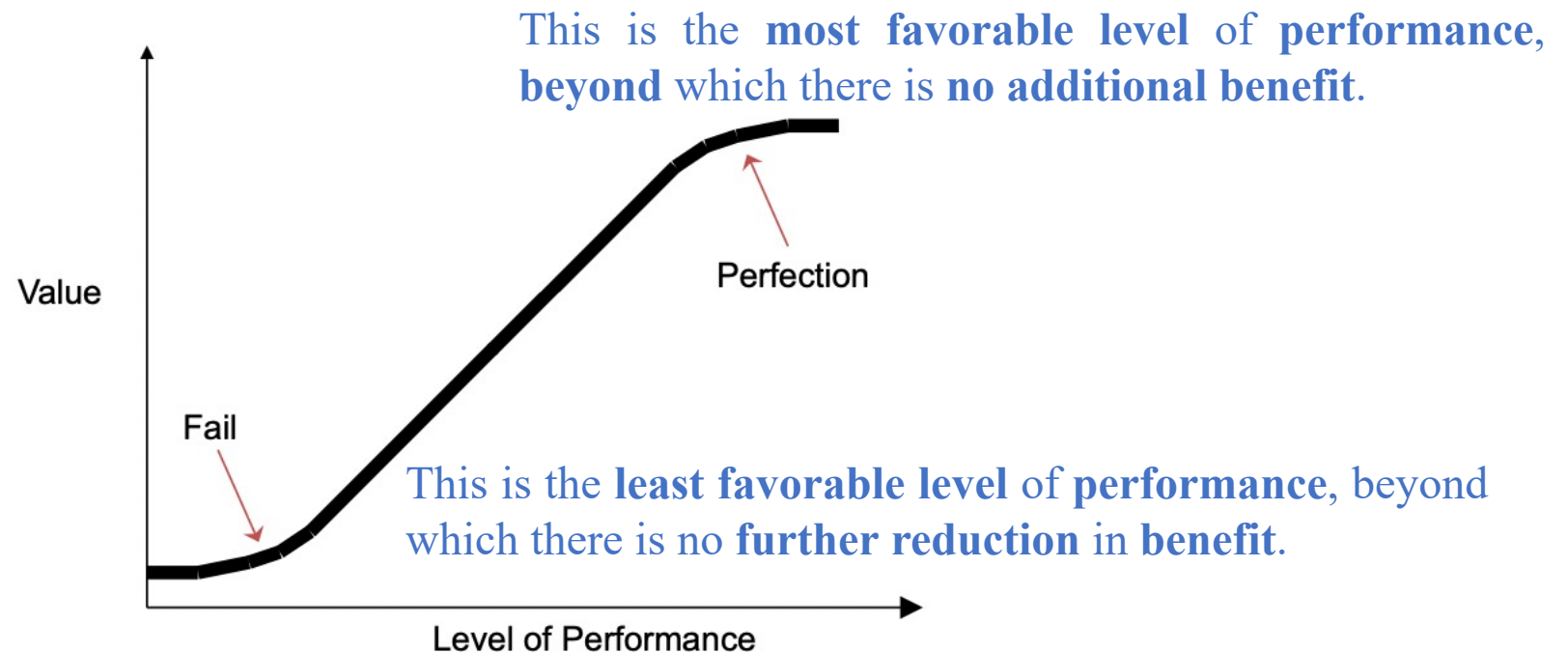
Work breakdown item:
3.2. Economics of Quality of Service Constraints

References used for this work breakdown item



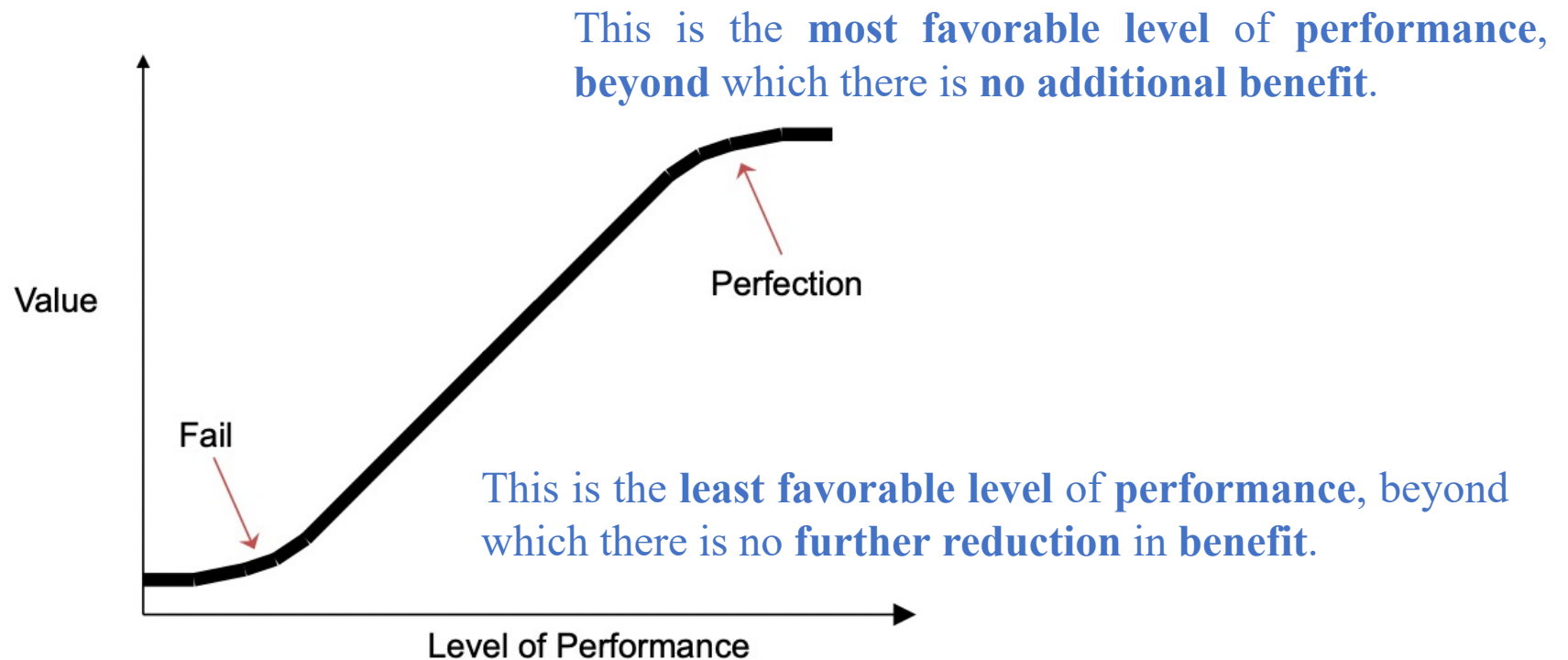
Economics of quality of service constraints

- The figure illustrates the economic perspective of a typical quality of service constraint where value increases with performance level.
 - ✓ Capacity, throughput, reliability, etc.



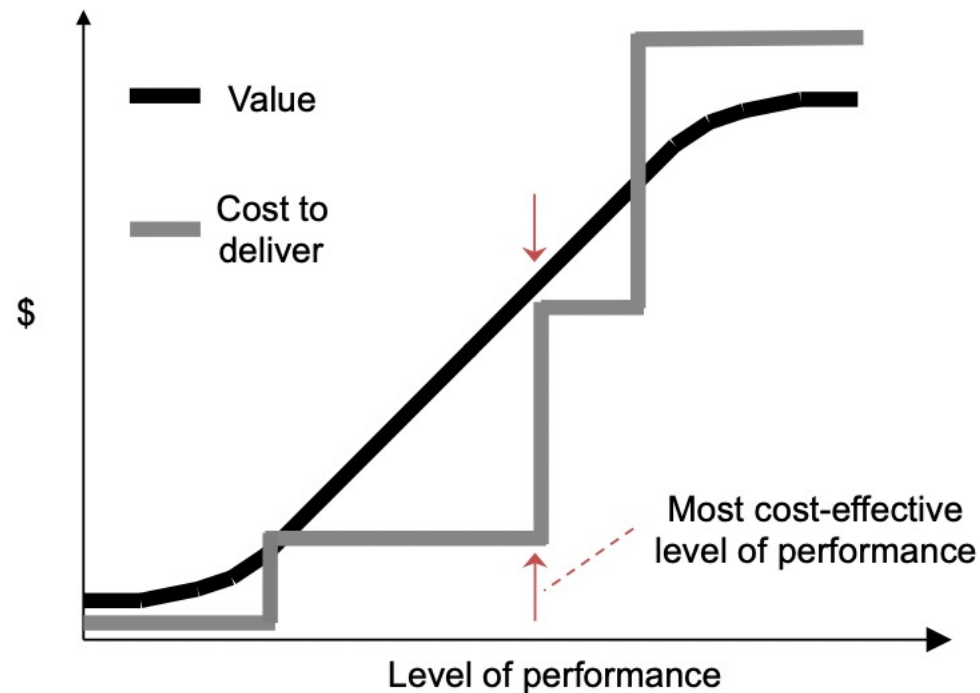
Economics of quality of service constraints

- This curve is mirrored vertically for quality of service constraints whose value decreases as performance level increases
 - ✓ Response time, mean time to repair, etc.



Economics of quality of service constraints

- The figure illustrates the **most cost-effective performance level**.
 - ✓ The **performance level** with the **maximum positive difference** between the **value** at that **performance level** and the **cost to achieve it**.



Work breakdown item:

3.3. Formal Analysis

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis 3.3
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Distance Based Requirements Specification
- Model Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

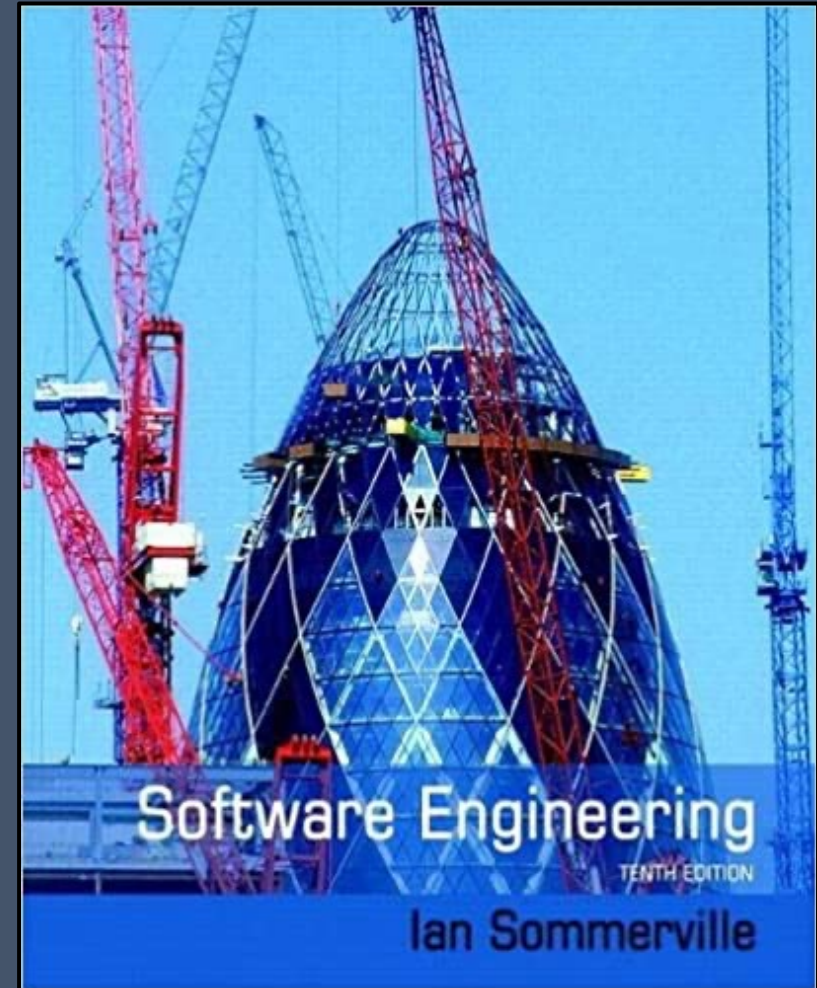
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
3.3. Formal analysis

References used for this work breakdown item



Sections 12.3.2 and 12.3.3

Formal analysis

- The **formal expression** of **requirements** depends on the **use** of a **specification language** with **formally** defined **semantics**.
- **Formal requirements** has **two benefits**:
 1. **Precise** and **concise**, which (in principle) will **reduce** the **possibility** for **misinterpretation**.
 2. Can be **reasoned over**, permitting **desired properties** of the specified **software** to be proved.

The formal analysis topic is related to Formal Methods in the Software Engineering Models and Methods KA.

Work breakdown item:

3.4. Addressing Conflict in Requirements

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
 - Why Categorize Requirements This Way?
 - System Requirements and Software Requirements
 - Derived Requirements
 - Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

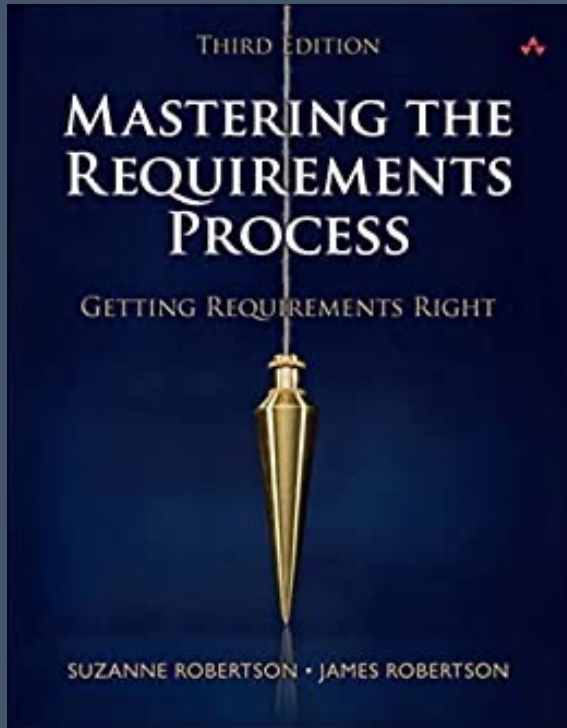
- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

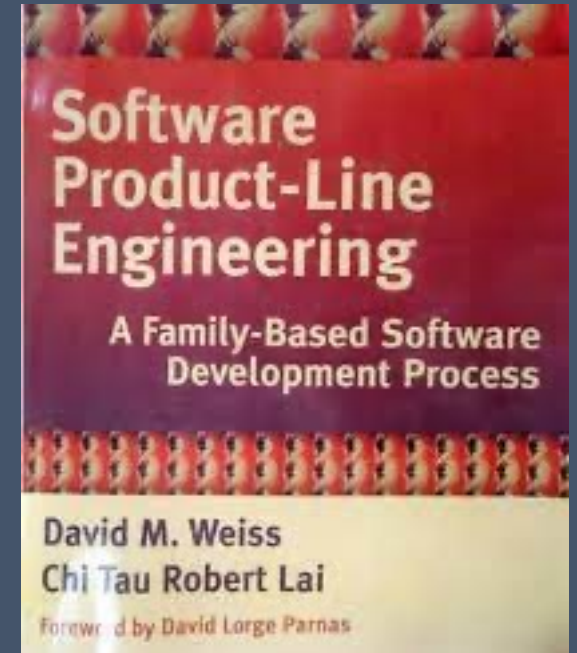
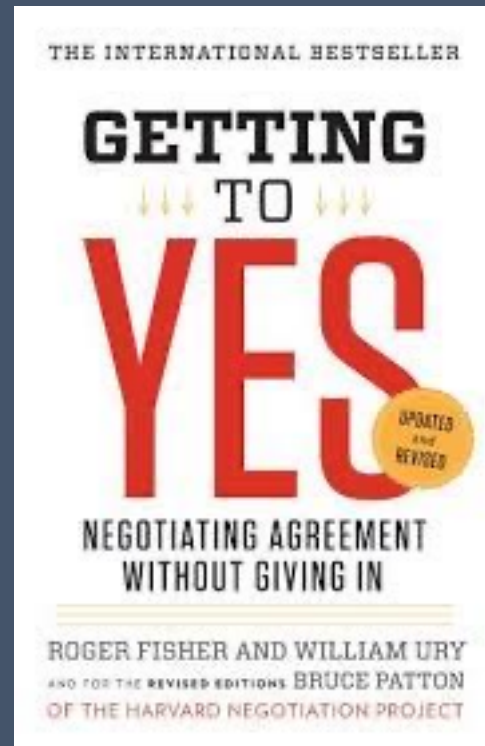
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
3.4. Addressing conflict in requirements

References used for this work breakdown item



Chapter 17



Addressing conflicts in requirements

- When a **project** has **more** (and more diverse) **stakeholders**, **conflicts among the requirements** are **more likely**.

Addressing conflicts in requirements

- The engineer may consider at least **two different approaches to managing conflict requirements** and **determine the most appropriate course of action**
- **Two possible approaches:**
 1. **Negotiate a resolution** among the **conflicting stakeholders**.
 2. **Apply product family development**.

The approach product family development

- Separates requirements into two categories.
 1. **Invariant** requirements.
 - Requirements that all stakeholders agree on.
 2. **Variant** requirements
 - Where **conflict exists**.

The approach product family development

- The **software** can be **designed** using:
 - ✓ **Design to invariants** to **accommodate** the **invariant requirements**
 - ✓ **Design for change** to **incorporate customization points** to **configure** an **instance** of the **system** to **best fit** relevant **stakeholders**.

Work Breakdown item:

4. Requirements specification

Software Requirements

4

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification 4.1
- Structured Natural Language Requirements Specification 4.2
- Acceptance Criteria-based Requirements Specification 4.3
- Model-Based Requirements Specification 4.4
- Additional Attributes of Requirements 4.5
- Incremental and Comprehensive Requirements Specification 4.6

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
4. Requirements specification

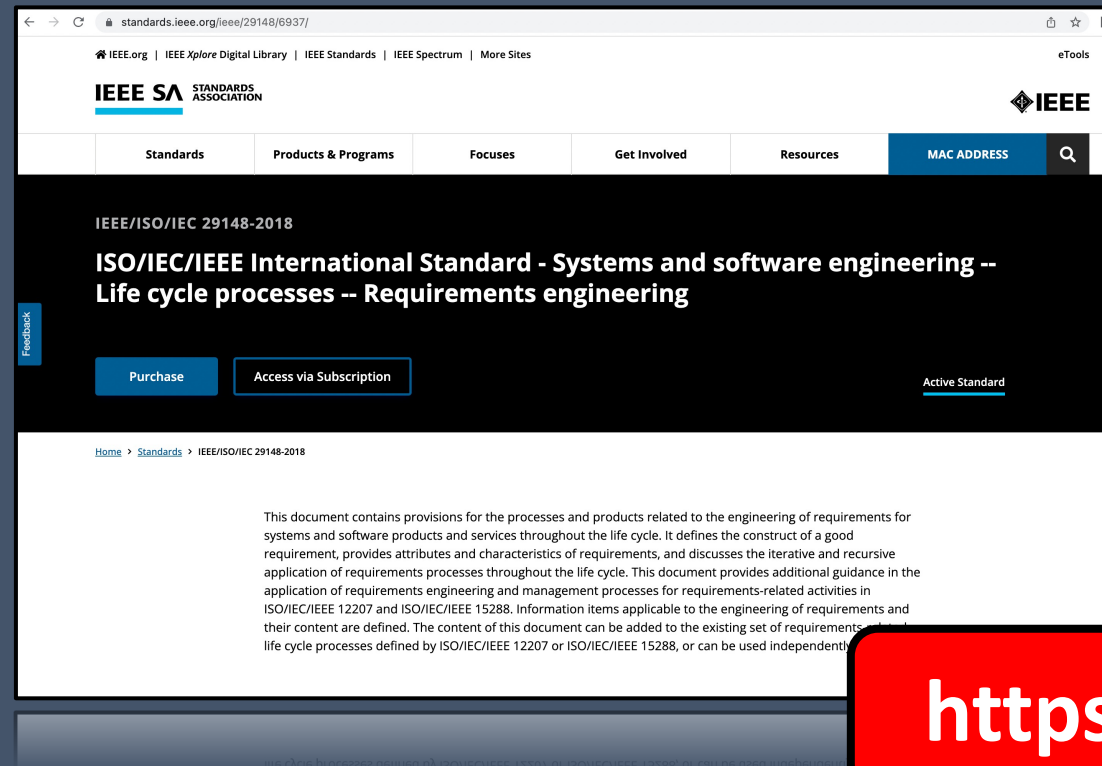
Introduction

Requirements specification

- **Requirements specification concerns recording the requirements:**
 - ✓ To be remembered and communicated.
- There are several general categories of requirements specification techniques.
- The requirements specification for a given project may also use various techniques.

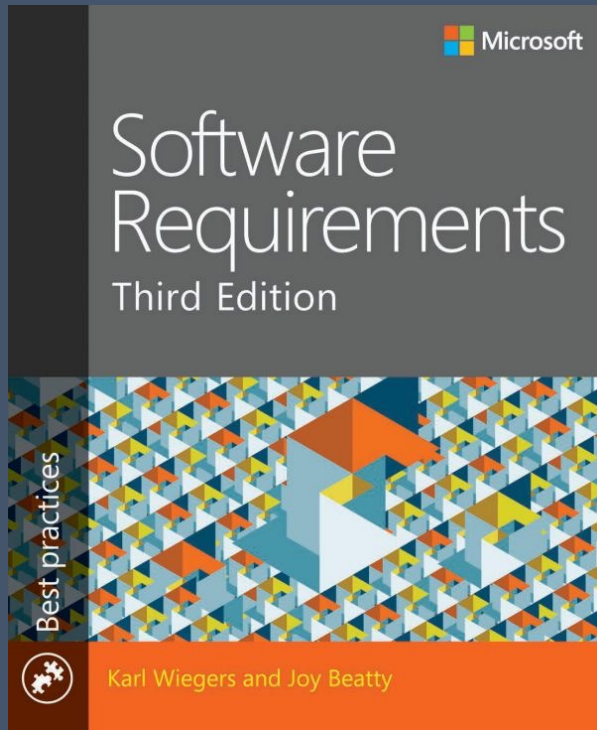
Templates for requirements documentation

- ISO/IEC/IEEE 29148 offer templates for requirements documentation.

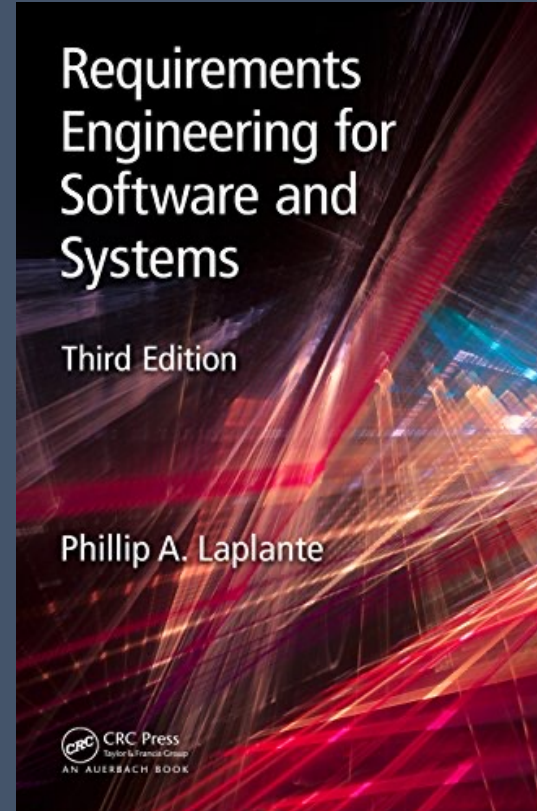


Templates for requirements documentation

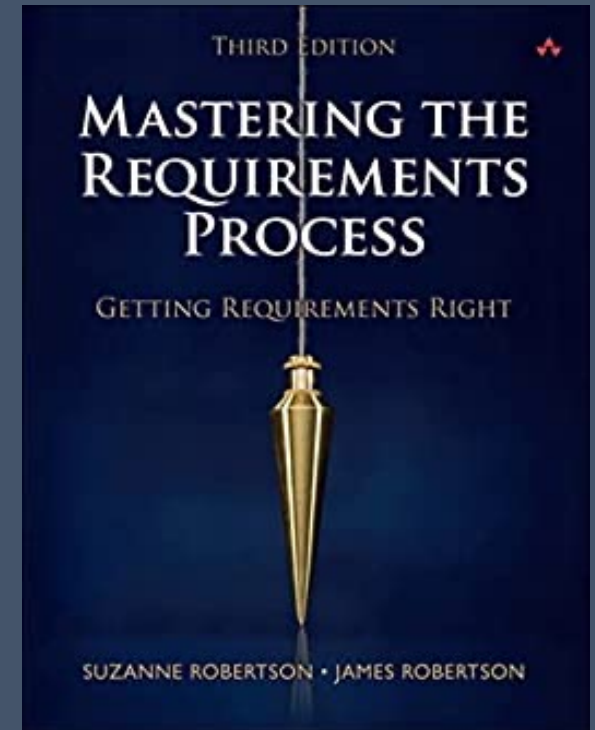
- Some example books that offer templates for requirements documentation.



Chapters 10-14



Chapter 4



Chapter 16

Work breakdown item:

**4.1. Unstructured Natural Language
Requirements Specification**

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:

4.1. Unstructured Natural Language Requirements Specification

- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Specification

Natural language requirements specification

- Express requirements in common, ordinary language.
- The specification can be:
 - ✓ Unstructured
 - ✓ Structured.

Unstructured natural language requirements

- A typical unstructured natural language requirements specification is a collection of statements in natural language.

“The **system** shall”

“The **system** shall”

“The **system** shall”

“The **system** shall”

Business rules and requirements

- **Business rules are statements that define or constrain some aspect of the structure or the behavior of the business to be automated.**
- **Example of a business rule that serves as a requirement for a university's course-registration software.**

“A student cannot register in next semester's courses if there remain any unpaid tuition fees”.

Work breakdown item:

**4.2. Structured Natural language
Requirements Specification**

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Constraint
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- **Structured Natural Language Requirements Specification**
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

4.2

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:

4.2. Structured Natural Language Requirements Specification

Structured natural language requirements

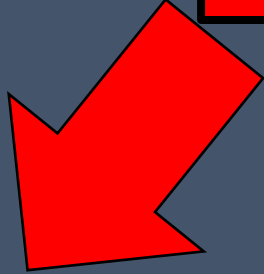
- Impose **constraints** on **how** the **requirements** are **expressed**.
- The **goal** is to **increase precision** and **conciseness**.

Actor-action

- The **simplest example of structured natural language format**:
 - ✓ The **actor** is the **entity** responsible for **carrying out the action**.
 - ✓ The **action** is **what needs to happen**.
 - ✓ A **triggering event** might **precede the actor**
 - ✓ The **action** might be **followed** by an **optional condition** or **qualification**.

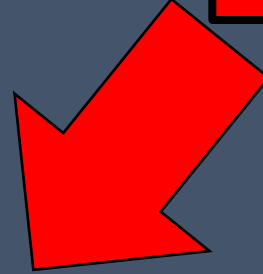
Examples of structured natural language requirements

Triggering event



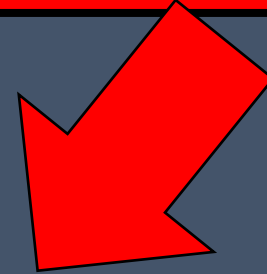
*“When an **order is shipped**, the system shall create an Invoice unless the Order Terms are ‘Prepaid’”.*

Actor



*“When an **order** is shipped, **the system** shall create an **Invoice** unless the **Order Terms** are ‘Prepaid’”.*

The action



*“When an **order** is shipped, the **system** shall **create an Invoice** unless the **Order Terms** are ‘Prepaid’”.*

*“When an **order** is shipped, the **system** shall create an **Invoice** unless the **Order Terms** are ‘Prepaid’”.*



The condition/qualification

Statement examples in actor-action format

Use case #66	Use case name: Reserve flight(s)
Triggering event(s)	Customer requests reservation(s) on flight(s)
Parameters	Passenger, itinerary, fare class, payment method(s)
Requires	Legal itinerary, fare class restrictions met
Guarantees	Seat(s) reserved for passenger on itinerary flight(s)
Normal course	Non-FF passenger, all domestic itinerary, Economy fare class, credit/debit card
Alternative course(s)	Is FF passenger: [None, Silver, Gold, Platinum, Elite] Itinerary: [all international, mixed domestic + international] Fare class: [Basic economy, Premium Economy, Business, First] Payment method: [Voucher, FF miles]
Exceptions	C/D card declined, voucher doesn't exist, voucher expired, FF account doesn't exist, insufficient miles in FF account

Statement examples in actor-action format

“As a <user> I want <capability> so that <benefit>”

Work breakdown item:

**4.3. Acceptance Criteria-based
Requirements Specification**

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification
- Additional

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

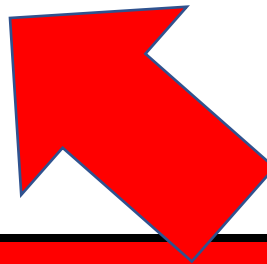
Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

4.3



Work breakdown item:

4.3. Acceptance Criteria-based Requirements Specification

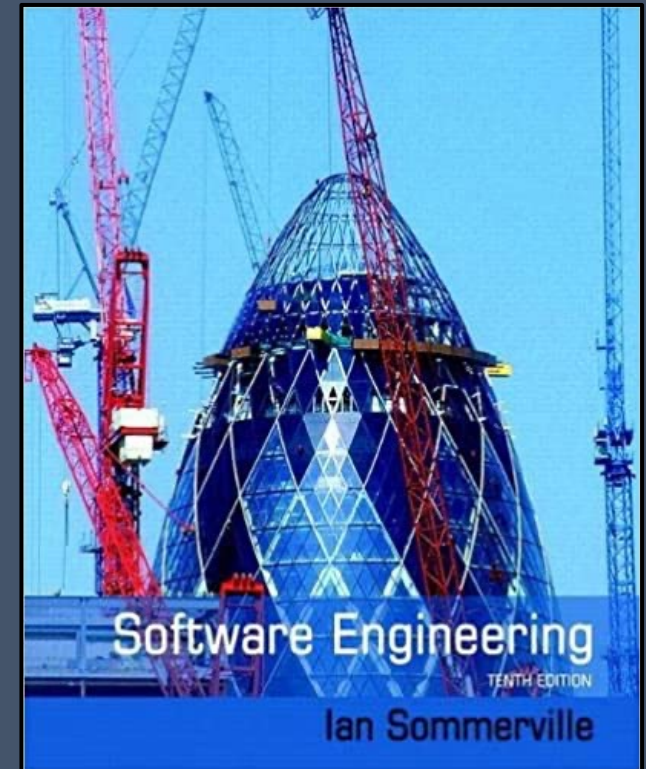
Approach variants

- The approach **Acceptance Criteria-Based Requirements Specification** includes **two** specific **variants**:
 1. **Acceptance Test Driven Development (ATDD)**
 2. **Behavior Driven Development (BDD).**

ATDD

Acceptance Test Driven Development - ATDD

- ATDD is a part of the larger Test Driven Development (TDD) approach.



Sections 3.2.3, 8.2 10-14

Test Driven Development (TDD)

- The **main** idea of **TDD** is that **test cases precede construction**.
- **No new production code is written and no existing code is modified unless at least one test case fails.**
 - ✓ Either at the **unit test level** or at the **acceptance test level**.

The 3 process steps in ATDD

1. A unit of functionality (e.g., a user story) is selected for implementation.

The 3 process steps in ATDD

2. Software engineers, business domain experts, QA/test professionals agree on a set of test cases that must pass to show that the unit of functionality has been correctly implemented.

The 3 process steps in ATDD

3. **At least one of the acceptance test cases must fail on the existing software giving the software engineer(s) permission to create or modify production code to pass all of the agreed-upon test cases.**

This **step** might require **several iterations**. The **code** may also be **refactored** during this **step**. When all **acceptance test cases** have **passed**, and **presumably** all **unit** and **integration test cases** as well, then the **unit of functionality** is **deemed** to have been **completely** and **correctly implemented**.

The 3 process steps in ATDD

The **ATDD** process returns to step 1, where a new unit of functionality is selected, and the cycle repeats.

ATDD for requirements or testing?

- **ATDD** might seem to be a **testing technique** rather than a **requirements specification technique**.
- But it also can be seen as a **requirements technique**.

ATDD for requirements specification

- A test case has the general form of:

“When given input that looks like X, we expect the software to produce results that look like Y.”

- The key is “we expect the software to produce.”

“When given input that looks like X, the software shall produce results that look like Y,”

- What **first looked** like a **test case** now looks like a **requirement**.

Using ATDD for requirements specification

- One acceptance test case can encompass more than one single requirement.
- ATDD test cases are precise, unambiguous statements of requirements.

BDD

The BDD approach

- Slightly more structured.
- **Business domain experts typically prefer it over ATDD because it is less technical in appearance.**

The BDD approach

- In BDD, the **unit of functionality** is **described** as a **user story**:

“As a <role>, I want <goal/desire> so that <benefit>.”

- This leads to the **identification** and **specification** of a set of “scenarios” in this form:

“**Given** <some context> [and <possibly more context>], **when** <stimulus> **then** <outcome> [and <possibly more outcomes>].”

The BDD approach: an example - US

“As a bank customer, I want to withdraw cash from the Automated Teller Machine (ATM) so that I can get money without going to the bank”.

The BDD approach: an example – Scenario 1

*“As a **bank customer**, I want to **withdraw cash** from the **Automated Teller Machine (ATM)** so that I can **get money without** going to the **bank**”.*

- **Scenario #1:** “the account has a sufficient balance.”

*“**GIVEN** the **account balance** is **\$500**, and the customer’s bank **card** is **valid**, and the **ATM** contains **enough money** in its **cash box**, **WHEN** the **Account Holder** requests **\$100**, **THEN** the **ATM** should dispense **\$100** and the **account balance** should be **\$400**, and the customer’s bank **card** should be **returned**.”*

The BDD approach: an example – Scenario 2

*“As a **bank customer**, I want to **withdraw cash** from the **Automated Teller Machine (ATM)** so that I can **get money without** going to the **bank**”.*

- **Scenario #2: “the account has an insufficient balance”.**

*“**GIVEN** the **account balance** is \$50, and the **customer’s bank card** is **valid**, and the **ATM** contains **enough money** in its cash box, **WHEN** the **Account Holder** requests \$100, **THEN** the **ATM** should **not dispense any money**, and the **ATM** should **say** there is an **insufficient balance**, the **balance** should **remain** at \$50, and the **customer’s bank card** should **be returned**.”*

The goal of BDD

- To have a **comprehensive set of scenarios** for each unit of **functionality**.
- **Additional scenarios** for the **withdrawing cash example**:

“The Bank Customer’s bank card has been disabled”

“The ATM does not contain enough money in its cash box”

...

...

The acceptance test cases are obvious
from the **BDD** scenarios.

Requirements ambiguity

Solving the requirements ambiguity problem

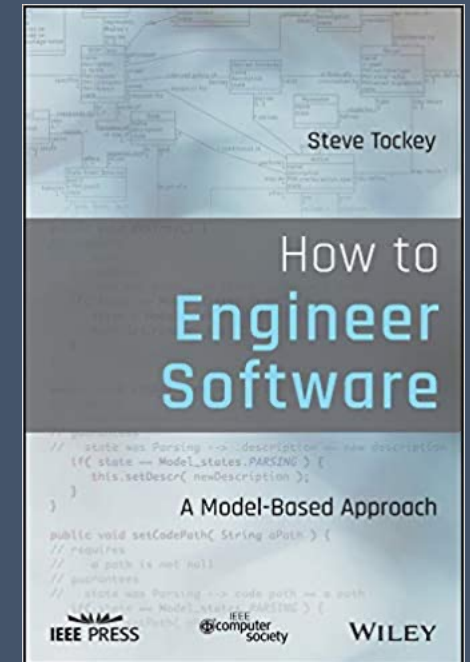
- Acceptance criteria-based requirements specification directly addresses the requirements ambiguity problem.
 - ✓ Natural languages are inherently ambiguous.
 - ✓ Test case language is not inherently ambiguous.

Writing requirements with test case language

- In acceptance-based criteria requirements specification, the requirements are written using test case language
 - ✓ Very precise.

Reduce requirements incompleteness

- Combining ATDD or BDD with appropriate functional test coverage criteria, can reduce the likelihood of requirements incompleteness.
 - ✓ Domain Testing
 - ✓ Boundary Value Analysis
 - ✓ Pairwise Testing
 - ✓ ...



Chapters 1 and 12

More information

- See the Software Testing KA.

CHAPTER 5		SOFTWARE TESTING	
ACRONYMS		HIPAA	Health Insurance Portability and Accountability Act
AI	Artificial Intelligence	HL7	Health Level Seven
API	Application Program Interface	IoT	Internet of Things
ARINC	Aeronautical Radio Incorporated	KPI	Key Performance Indicator
ATDD	Acceptance Test-Driven Development	MC/DC	Modified Condition Decision Coverage
CMM	Capability Maturity Model	ML	Machine Learning
CSS	Cascading Style Sheets	MTTR	Mean Time to Recovery
DICOM	Digital Imaging and Communications in Medicine	OAT	Orthogonal Array Testing
DL	Deep Learning	ODC	Orthogonal Defect Classification
DU	Definition and Use	RUP	Rational Unified Process
EBSE	Evidence-Based Software Engineering	SoS	System of Systems
ECS	Ecosystem	SPI	Software Process Improvement
ETSI	European Telecommunications Standards Institute	SPICE	Software Process Improvement and Capability Determination
FHIR	Fast Healthcare Interoperability Resources	SUT	System Under Test
GDPR	General Data Protection Regulation	TDD	Test-Driven Development
GPS	Global Positioning System	TMMi	Test Maturity Model integration
GUI	Graphical User Interface	UI	User Interface

Source: <https://bit.ly/3y3DoW8>

Work breakdown item:

4.4. Model-Based Requirements Specification

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
 - Software Product Requirements and Software Project Requirements
 - Functional Requirements
 - Nonfunctional Requirements
 - Technology Constraints
 - Quality of Service Constraints
 - Why Categorize Requirements This Way?
 - System Requirements and Software Requirements
 - Derived Requirements
 - Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Communication Techniques

Constraints

- Formal Analysis
- Addressing Conflict in Requirements

Structured Natural Language Requirements Specification

- Acceptance Criteria-based Requirements Specification
- **Model-Based Requirements Specification**
- ~~Additional~~
- Attributes of Requirements
- Incremental and Comprehensive Requirements Specification

4.4

Scope Matching

- Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
4.4. Model-Based Requirements Specification**

Modeling languages

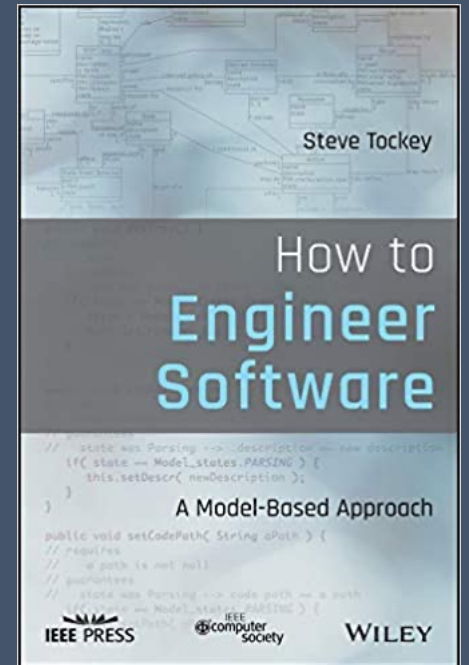
- **Another approach to avoiding the inherent ambiguity of natural languages is to use modeling languages.**
- **Examples:**
 - ✓ **Selected elements of the Unified Modeling Language (UML).**
 - ✓ **Selected elements of the Systems Modeling Language (SysML).**

The two categories of requirements models

1. **Structural** models
2. **Behavioral** models

Structural models

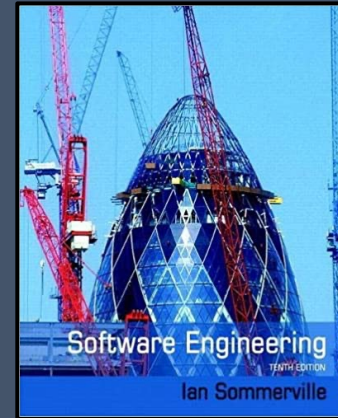
- Specify policies to be enforced:
- Logical class models.
 - ✓ Also called conceptual **data model**, logical **data models** and **entity-relationship** diagrams.



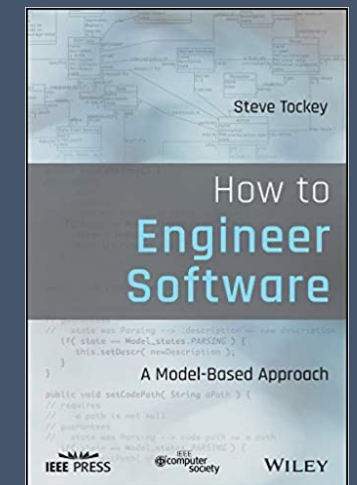
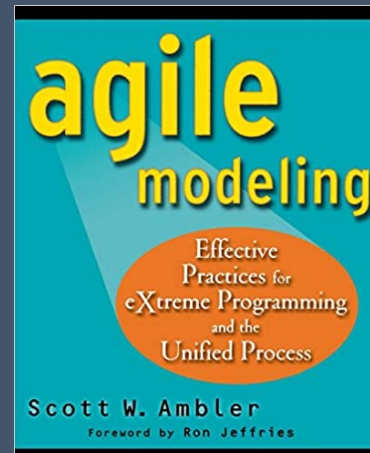
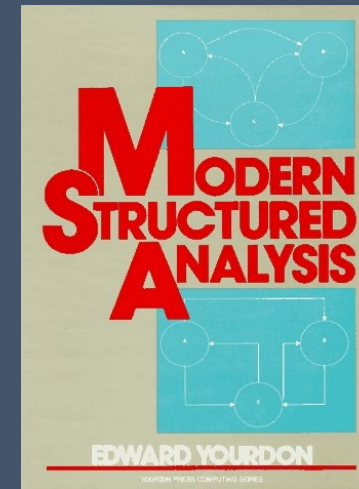
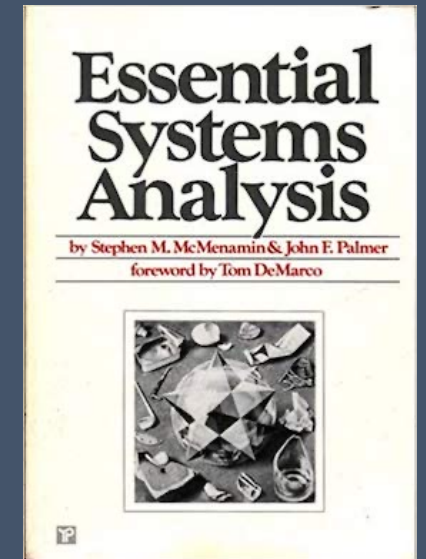
Chapter 8

Behavioral models

- Specify processes to be carried out.
- Examples:
 - ✓ Use case modeling.
 - ✓ Interaction diagrams and state modeling.
 - ✓ UML activity diagrams.
 - ✓ Data-flow modeling.



Chapters 12, 13



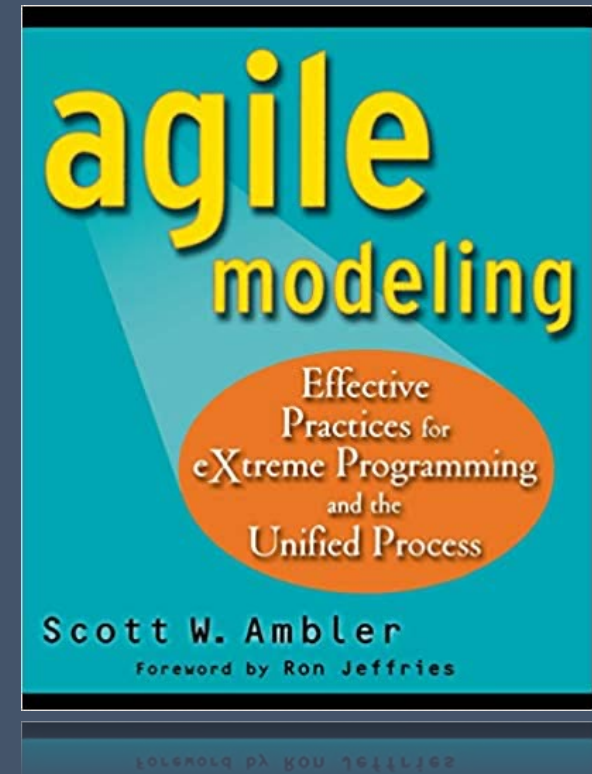
Chapters 7, 9, 10

Degree of requirements model formality

- Model-based requirements specifications vary in the degree of model formality:
 - ✓ Informal modeling
 - ✓ Semiformal modeling
 - ✓ Formal modeling

Degree of requirements model formality

- Agile modeling is the least formal (informal).



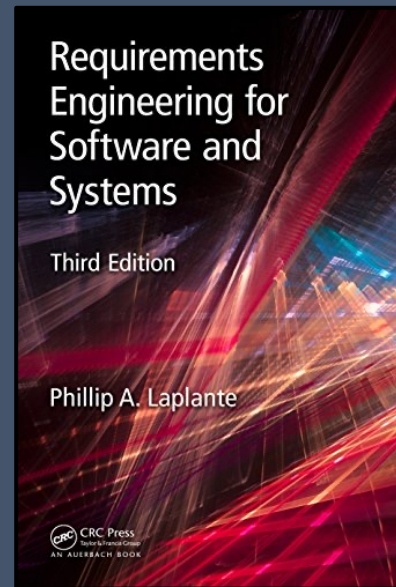
Degree of requirements model formality

- Semiformal modeling

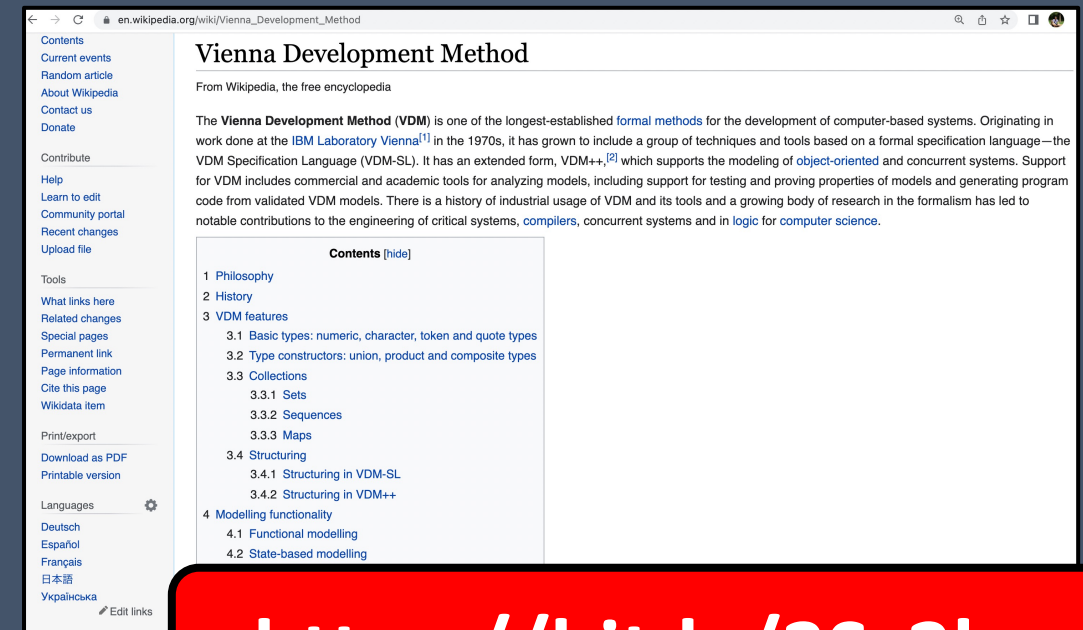
- ✓ Provides a definition of the modeling language semantics.
- ✓ However, the definition has not been formally proved to be complete and consistent.

Degree of requirements model formality

- **Formal modeling languages with precisely defined semantics:**
 - ✓ **Z**, the **Vienna Development Method (VDM)**, **Specification and Description Language (SDL)**.



Chapter 6



<https://bit.ly/3Ss2bva>

Degree of requirements model formality

- The defined semantics allow specifications to be mechanically analyzed for the presence or absence of specific properties
 - ✓ Help avoid critical reasoning errors.

Benefits of more formal requirements

- **Generally, the more formal a requirements model is, the less ambiguous it is.**
 - ✓ **Software engineers are less likely to misinterpret the requirements.**
- **More formal requirements models can also be:**
 - ✓ **More concise and compact.**
 - ✓ **Easier to translate into code, possibly mechanically.**
 - ✓ **Used as a basis for deriving acceptance test cases.**

Work breakdown item:

4.5. Additional Attributes of Requirements

Software Requirements

Work breakdown item: 4.5. Additional Attributes of Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

- Formal Analysis
- Addressing Conflict in Requirements

- Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification
- Additional Attributes of Requirements
- Incremental and Comprehensive Requirements Specification

4.5

Requirements

Matching

- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Additional attributes of requirements

- In addition to basic requirements statements, documenting additional attributes for some or all requirements can be useful.

Examples of additional attributes

- **Tag**
 - ✓ To support **requirements tracing**.
- **Description**
 - ✓ Additional **details** about the **requirement**.
- **Rationale**
 - ✓ **Why** the **requirement** is **important**.
- **Source**
 - ✓ **Role** or **name** of the **stakeholder** who **imposed** this **requirement**.

Examples of additional attributes (cont.)

- Use case or relevant triggering event.
- Type
 - ✓ E.g., functional, quality of service.
- Dependencies
- Conflicts

Examples of additional attributes (cont.)

- Acceptance criteria.
- Priority
- Stability
- Whether the requirement is common or a variant for product family development.
- Supporting materials.
- Requirement's change history.

Work breakdown item:

**4.6. Incremental and Comprehensive
Requirements Specification**

Software Requirements

Software

Requirements

Requirements

Requirements

Requirements

Requirements

Practical

Software

Work breakdown item:

4.6. Incremental and Comprehensive Requirements Specification

Requirements and
Software Project
Requirements

Functional
Requirements

Nonfunctional
Requirements

Technology
Constraints

Quality of Service
Constraints

Why Categorize
Requirements
This Way?

System Requirements
and
Software Requirements

Derived
Requirements

Software Requirements
Activities

Analysis

Addressing
Conflict in
Requirements

Language
Requirements
Specification

Acceptance
Criteria-based
Requirements
Specification

Model-Based
Requirements
Specification

Additional
Attributes of
Requirements

Incremental and
Comprehensive
Requirements
Specification

4.6

Tracing

Requirements
Stability and
Volatility

Test Case
Generation
Tools

Approaches to document requirements

- Projects that explicitly document requirements take one of two approaches:
 1. Incremental specification.
 2. Comprehensive specification.

Incremental requirements specification

- A version of the requirements specification contains only the differences from the previous version.
 - ✓ Additions, modifications and deletions.
- An advantage of this approach is that it can produce a smaller volume of written specifications.

Comprehensive requirements specification

- Each version's **requirements specification** contains **all requirements**.
 - ✓ **Not just changes** from the **previous version**.
- An **advantage** of this **approach** is that a **reader** can **understand** all **requirements** in a **single document**
 - ✓ Instead of having to keep **track** of **cumulative additions**, **modifications** and **deletions** across a **series** of **specifications**.

Hybrid requirements specification

- Some **organizations combine** these **two approaches**
 - ✓ **Intermediate releases** specified incrementally.
 - E.g., x.1, x.2 and x.3.
 - ✓ **Major releases** that are specified comprehensively.
 - E.g., 1.0, 2.0 and 3.0.
- The **reader never needs** to go any **further back** than the **requirements specifications** for the **last major release** to **obtain** the **complete set of specifications**.

Breakdown item:

5. Requirements validation

Software Requirements

5

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Access Control Requirements Specification

Requirements Validation

- Requirements Reviews 5.1
- Simulation and Execution 5.2
- Prototyping 5.3

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
5. Requirements validation

Introduction

Requirements validation

- Concerns gaining confidence that the requirements represent the stakeholders' true needs as they are currently understood (and possibly documented).



Key questions

- Do these **represent all requirements relevant at this time?**
- Are any stated **requirements not representative of stakeholder needs?**
- Are these **requirements appropriately stated?**
- Are the **requirements understandable, consistent and complete?**
- Does the **requirements documentation conform to relevant standards?**

3 methods for requirements validation

1. Requirements reviews
2. Simulation and execution
3. Prototyping

Work breakdown item:

5.1. Requirements reviews

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Access Control Requirements Specification

Requirements Validation

- Requirements Reviews **5.1**
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
5.1. Requirements Reviews**

Validate a requirements document

- The most common way to **validate** is by **reviewing** or **inspecting** a **requirements document**.
- **One or more reviewers** are asked to look for:
 - ✓ **Errors**
 - ✓ **Omissions**
 - ✓ **Invalid assumptions**
 - ✓ **Lack of clarity**
 - ✓ **Deviation from accepted practice**

Favor reviews from multiple perspectives

- Clients, customers and users.
- Software engineers
 - ✓ With expertise in requirements specification.
- Software engineers
 - ✓ Who will do architecture, design or construction of the software.

Work breakdown item:

5.2. Simulation and execution

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Accessing Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution 5.2
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
5.2. Simulation & execution

Simulation and execution

- **Nontechnical stakeholders might not want to spend time reviewing a specification in detail.**
- **Some specifications can be subjected to simulation or actual execution in place of or in addition to human review.**

Simulation and execution

- To the **extent** that the **requirements** are **formally specified**, **software engineers** can **hand interpret** that **specification** and “**execute**” the **specification**.
 - ✓ E.g., in a **model-based specification**.
- Given a **sufficient set** of **demonstration scenarios**, **stakeholders** can be **convinced** that the **specification** defines their **policies** and **processes completely** and **accurately**.

Work breakdown item:

5.3. Prototyping

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Accessing Requirements Specifications

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
5.3. Prototyping

Prototyping

- If the requirements can't be directly simulated, an alternative is to create a prototype that demonstrates some important dimension of an implementation.
- This demonstrates the software engineer's interpretation of the requirements.

Prototyping: example

- A user interface's dynamic behavior might be better understood through an animated prototype than through textual description or graphical models.
- A danger of prototyping is that cosmetic issues or quality problems with the prototype can distract the reviewers' attention from the core underlying functionality.

Breakdown item:

6. Requirements management activities

Software Requirements

6

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing 6.1
- Requirements Change Control 6.2
- Scope Matching 6.3

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

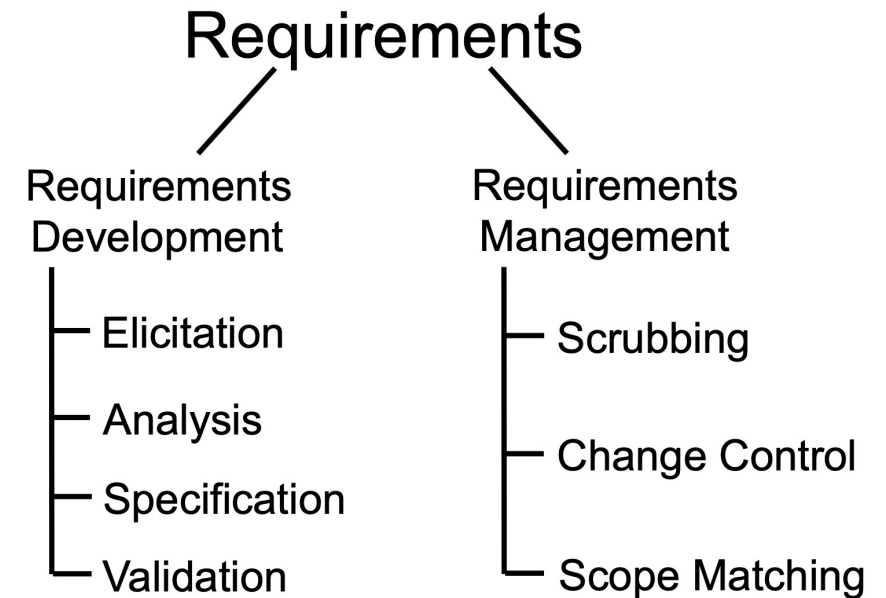
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
6. Requirements management activities**

Introduction

Requirements

- **Requirements development.**
“Reaching an agreement on what software is to be constructed.”
- **Requirements management.**
“Maintaining that agreement over time.”



Work breakdown item:

6.1. Requirements Scrubbing

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing **6.1**
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
6.1. Requirements Scrubbing**

- Requirements
- Incremental and Comprehensive Requirements Specification

Requirements scrubbing

- Practice in which a **product specification** is **carefully examined** for **unnecessary** or **overly complex requirements**.
 - ✓ Which are then removed.
- The **goal of requirements scrubbing** is to **find the smallest set of simply stated requirements** that will **meet stakeholder needs**.
- Doing so will **reduce the size and complexity** of the **solution**
 - ✓ Minimizing the **effort, cost** and **schedule** to deliver it.

Requirements scrubbing

- Requirements scrubbing involves eliminating requirements that:
 - ✓ are out of scope;
 - ✓ would not yield an adequate return on investment;
 - ✓ are not that important.
- Another important part of the process is to simplify unnecessarily complicated requirements.



Source: <https://tinyurl.com/8pmpxdu9>

Requirements scrubbing in waterfall and agile

Requirements scrubbing

- **Waterfall and other plan-based life cycles:**
 - ✓ Requirements scrubbing can be coordinated with requirements reviews for validation.
 - ✓ Scrubbing should occur just before the validation review.
- **Agile life cycles:**
 - ✓ Scrubbing happens implicitly in iteration planning.
 - ✓ Only the highest-priority requirements are brought into a sprint (iteration).

Work breakdown item:

6.2. Requirements Change Control

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
6.2. Requirements Change Control**

- Requirements
- Incremental and Comprehensive Requirements Specification

Change control is central to managing requirements.

Requirements change control in waterfall and agile

Requirements change control: waterfall

- An **explicit requirements change control process** must include:
 - ✓ A means to **request changes to requirements**.
 - ✓ Optional **impact analysis stage** to examine **benefits** and **costs** of a **requested change**.
 - ✓ A **responsible person or group** who **decides to accept, reject, or defer** each **requested change**.
 - ✓ A means to **notify** all affected **stakeholders** of that **decision**.
 - ✓ A means to **track accepted changes** to **closure**.

Requirements change control: agile

- Requirements change management happens implicitly in Agile life cycles.
 - ✓ Any request to change requirements becomes just another item on the product backlog.
 - ✓ A request will only become “accepted” when it is prioritized highly enough to make it into an iteration/sprint.

Work breakdown item:

6.3. Scope Matching

Software Requirements

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching **6.3**

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
6.3. Scope Matching

Scope matching

- **Involves ensuring that the scope of requirements to architect, design and construct does not exceed:**
 - ✓ **Cost**
 - ✓ **Schedule**
 - ✓ **Staffing constraints on the project.**

What if scope matching exceed constraints?

- When requirements scope exceeds the cost, schedule or staffing constraints, either:
 - ✓ The scope must be reduced:
 - By removing a sufficient number of the lowest-priority requirements.
 - ✓ Capacity must be increased:
 - By extending the schedule or increasing the budget and/or staffing.
 - ✓ Some combination thereof must be negotiated.

Scope matching in waterfall and agile

Scope matching in waterfall (plan-based)

- **Scope matching** can be **coordinated** with **requirements validation**.
- **Scope matching** should **occur** just **before** the **validation review**.

Scope matching in agile

- The **only work allowed** into a **sprint/iteration** will be the **work** that can **reasonably be expected** to be **completed** during that **sprint/iteration**.
 - ✓ As long as **some variant of velocity-based sprint planning** is **done**.

Breakdown item:

7. Practical considerations

Software Requirements

7

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process **7.1**
- Requirements Prioritization **7.2**
- Requirements Tracing
- Requirements Stability and Volatility **7.3**

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
7. Practical considerations

Breakdown item:

7.1. Iterative nature of the requirements process

Software Requirements

7

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process **7.1**
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
7.1 Iterative nature of the requirements process

Evolutionary requirements

- **Requirements for typical software** not only have **wide breadth**.
 - ✓ They must also have **significant depth**.
- The **tension** created by **simultaneous breadth-wise and depth-wise requirements** in **real-world projects** often **prompts** teams to perform **requirements activities iteratively**.

Evolutionary requirements

- At **some points**, **elicitation** and **analysis** favor expanding the **breadth** of **requirements** knowledge.
 - ✓ While at **other points**, expanding the **depth** is called for.
- In **practice**, it is **highly unlikely** that **all requirements work** can be **done** in a **single pass** through the **subject matter**.

Breakdown item:

7.2. Requirements prioritization

Software Requirements

7

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
7.2. requirements prioritization**

Specification

Requirements prioritization usefulness

- **Prioritizing requirements is useful throughout a software project.**
 - ✓ Because it helps focus software engineers on delivering the most valuable functionality soonest.

Prioritization schemes

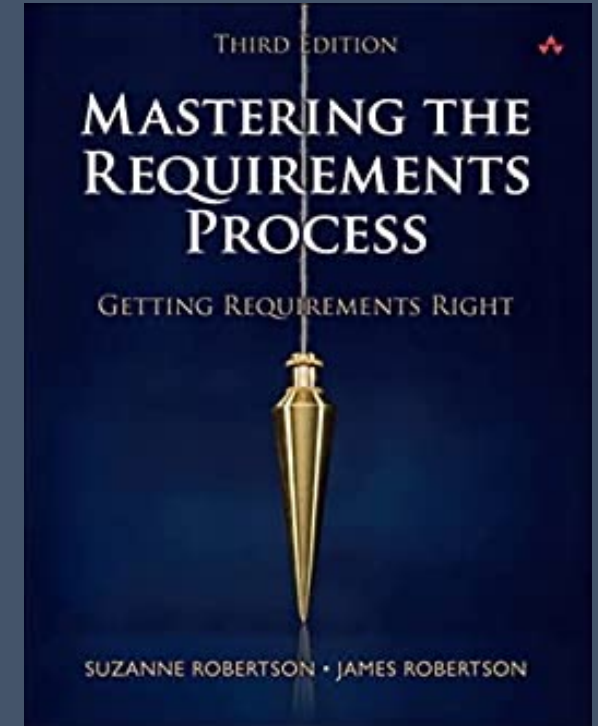
- A **variety of prioritization schemes** are available.
 - ✓ Answering a few key questions can help engineers choose the best approach.
- The **first key question** is:
“What factors are relevant in determining the priority of one requirement over another?”
- The **second key question** is:
“How can we convert the set of relevant factors into an expression of priority?”

What factors are relevant to prioritization?

- The following factors might be relevant to a project:
 - ✓ Value
 - ✓ Desirability
 - ✓ Client, customer and user satisfaction
 - ✓ Undesirability
 - ✓ Client, customer and user dissatisfaction
 - Kano model
 - ✓ Cost to deliver
 - ✓ Cost to maintain over the software's service life
 - ✓ Technical risk of implementation
 - ✓ Risk that users will not use it even if implemented.

The Kano model

- Shows that **considering only value, desirability or satisfaction** can lead to **erroneous priorities**.
- A **better understanding of priorities** comes from considering **how unhappy stakeholders would be if that requirement were not satisfied**.



Chapter 17

Weigh satisfaction and dissatisfaction

- **Prioritization must weigh both:**
 - ✓ The **satisfaction** users will **experience** from having certain **features**.
 - ✓ The **dissatisfaction** they will **experience** if they **lack** certain **features**.

Expression of priority

- The second key question is:

“How can we convert the set of relevant factors into an expression of priority?”

- The formula is just one example of an objective function to do so.

$$Priority = \frac{(Value * (1 - Risk))}{Cost}$$

Communicating requirements priority

- Once the **priority** of the **requirements** has been **determined**, those **priorities** must be **specified** in a **way** that can be **communicated** to all **stakeholders**.
- **Several ways are possible:**
 - ✓ **Enumerated scale**
 - E.g., must have, should have, nice to have.
 - ✓ **Numerical scale**
 - E.g., 1 . . . 10
 - ✓ **Lists** that **sort** the **requirements** in decreasing priority order.

Breakdown item:

7.3. Requirements tracing

Software Requirements

7

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
7.3. Requirements Tracing**

7.3

Purposes of requirements tracing

1. To documents consistency between pairs of related project work products.
2. Assist in **impact analysis** of a proposed **requirement change**.

First purposes of requirements tracing

1. Document consistency between pairs of related project work products.

“For each identified software requirement, are there identified design elements intended to satisfy it?”

“For each identified design element, are there identified requirements that cause it to exist?”

Second purpose of requirements tracing

2. Assist in impact analysis of a proposed requirement change.

The affected software requirements, design elements and code units could also be traced to their linked test cases for further impact analysis.

Requirements tracing

- **Software requirements** can be traced back to source documentation such as **system requirements**, **standards documents** and other relevant **specifications**.
- **Software requirements** can also be traced forward to design elements and **requirements-based test cases**.
- Finally, **software requirements** can also be traced forward to sections in a **user manual** describing the **implemented functionality**.

Breakdown item:

7.4. Requirements stability and volatility

Software Requirements

7

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Requirements This Way?
- System and Software Requirements
- Deriving Requirements from Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification
- Additional

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

7.4

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
7.4. Requirements Stability and Volatility

Requirements stability and volatility

- Some **requirements** are **very stable**.
 - ✓ Probably **never change** over the **software's service life**.
- Some **requirements** are **less stable**.
 - ✓ **Might change** over the **service life**, but might **not change** during the **development project**.
- **Identifying** potentially **volatile requirements** helps the **software engineer** establish a **design** more **tolerant of change**.

Disclaimer

- **Work breakdown itens on the text of the SWEBOK, but not in the graphical representation.**
 - ✓ Measuring requirements.
 - ✓ Requirements Process Quality and Improvement.
- **I edited the graphical representation to include it.**
 - ✓ See nex slides.

Breakdown item:

7.5. Measuring Requirements

Software Requirements

7

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification
- Additional Attributes of Requirements

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility
- Measuring Requirements 7.5
- Requirements Process Quality and Improvement

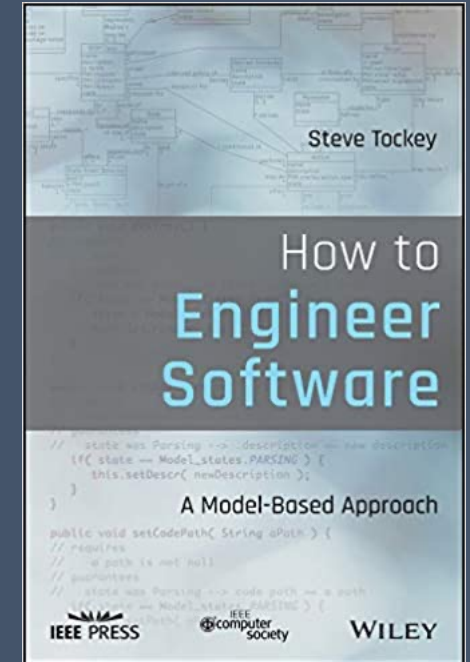
Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

Work breakdown item:
7.5. Measuring requirements

Measuring requirements

- It may be useful to have some concept of the volume of the requirements for a particular software product.
- This number is useful in:
 - ✓ Evaluating the size of a new development project or the size of a change in requirements.
 - ✓ Estimating the cost of development or maintenance tasks,
 - ✓ For use as the denominator in other measurements.



Chapter 23

Functional Size Measurement (FSM)

- Technique for evaluating the size of a body of functional requirements.

Quality indicators

- **Many quality indicators have been developed.**
- **Can be used to relate the quality of software requirements specification to other project variables such as:**
 - ✓ **Cost**
 - ✓ **Acceptance**
 - ✓ **Performance**
 - ✓ **Schedule**
 - ✓ **Reproducibility.**

Quality indicators

- **Quality indicators for individual software requirements and a requirements specification document as a whole can be derived from the desirable properties.**
 - ✓ Discussed in **Basic Requirements Analysis work breakdown item**, earlier in this KA.

Breakdown item:

7.6. Requirements Process Quality and Improvement

Software Requirements

7

Software Requirements Fundamentals

Requirements Elicitation

Requirements Analysis

Requirements Specification

Requirements Validation

Requirements Management Activities

Practical Considerations

Software Requirements Tools

Work breakdown item:

7.6. Requirements Process Quality and Improvement

- Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Addressing Conflict in Requirements

- Requirements Specification
 - Acceptance Criteria-based Requirements Specification
 - Model-Based Requirements Specification
 - Additional Attributes of Requirements
 - Incremental and Comprehensive Requirements Specification

Requirements Stability and Volatility

Measuring Requirements

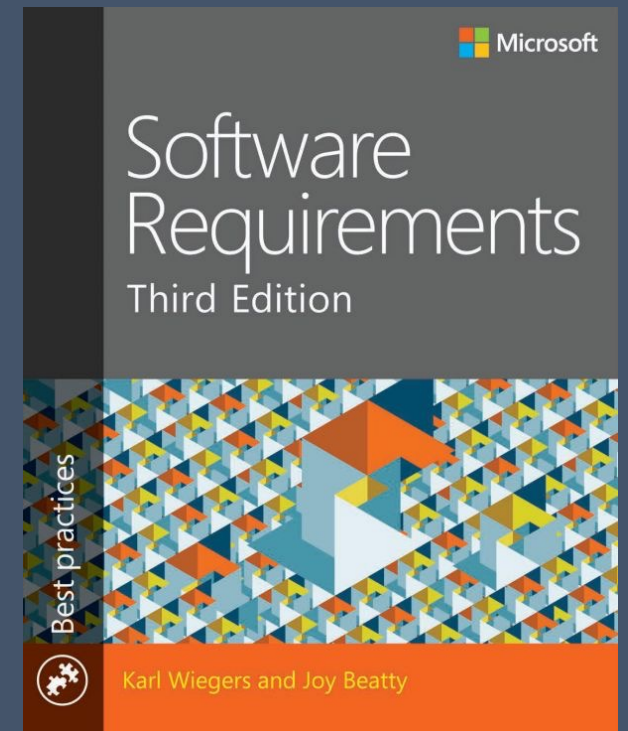
Requirements Process Quality and Improvement

7.6

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

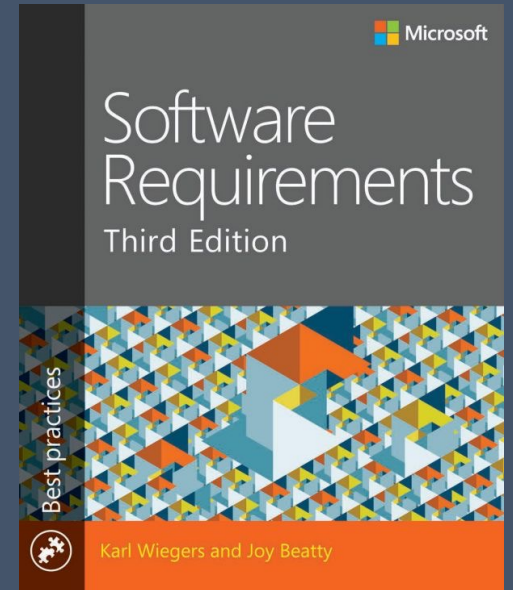
Requirements process quality and improvement

- Its purpose is to emphasize the key role of the requirements process in a software product's cost and timeliness and in customer satisfaction.
- Furthermore, it helps align the requirements process with quality standards and process improvement models for software and systems.



Requirements process quality and improvement

- It comprises the following:
 - ✓ Requirements process coverage by process improvement standards and models.
 - ✓ Requirements process measures and benchmarking.
 - ✓ Improvement planning and implementation
 - ✓ Security/CIA, improvement/planning, and implementation.
 - Confidentiality, Integrity, and Availability (CIA).



Breakdown item:

8. Software requirements tools

Software Requirements

8

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools **8.1**
- Requirements Modeling Tools **8.2**
- Functional Test Case Generation Tools **8.3**

**Work breakdown item:
8. Software Requirements Tools**

Introduction

Categories of software requirements tools

1. Requirements management tools.
2. Requirements modeling tools.
3. Functional test case generation tools.

Breakdown item:

8.1. Requirements management tools

Software Requirements

8

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Comprehensive Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirement Prioritization
- Requirement Tracing
- Requirement Stability and Volatility

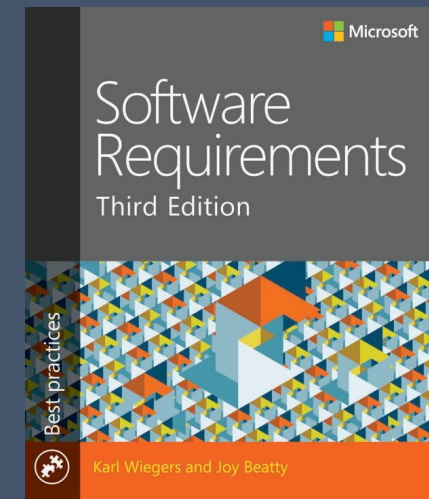
Software Requirements Tools

- Requirements Management Tools **8.1**
- Requirements Modeling Tools
- Functional Test Case Generation Tools

**Work breakdown item:
8.1. Requirements management tools**

Requirements Management Tools

- Requirements management tools support various activities:
 - ✓ Storing requirements attributes, tracing, document generation and change control.
- Tracing and change control might only be practical when supported by a tool.
- Many organizations have invested in tools.
 - ✓ However, many more manage their requirements in more ad hoc and generally less satisfactory ways
 - E.g., spreadsheets.



Breakdown item:

8.2. Requirements modeling tools

Software Requirements

8

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based Requirements Specification

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

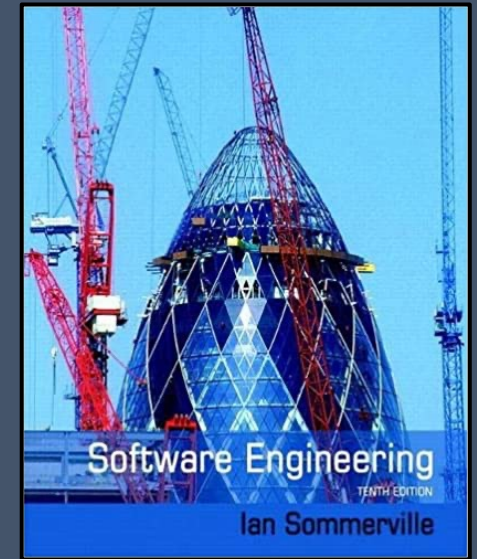
- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

8.2

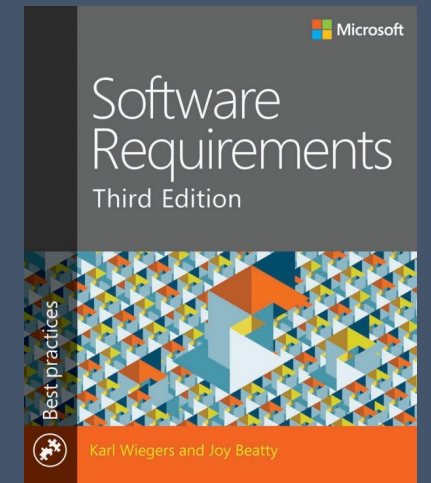
**Work breakdown item:
8.2. Requirements modeling tools**

Requirements modeling tools

- At a minimum, a requirements modeling tool supports creating, modifying and publishing model-based requirements specifications.
- Some tools extend that by also providing static analysis.
 - ✓ E.g., syntax correctness, completeness and consistency.
- Formal analysis requires tool support to be practicable.
 - ✓ Tools generally fall into two categories: theorem provers or model checkers.
- Some tools also dynamically execute a specification.
 - ✓ Simulation.



Section 12.3.3



Breakdown item:

8.3. Functional test case generation tools

Software Requirements

8

Software Requirements Fundamentals

- Definition of a Software Requirement
- Categories of Software Requirements
- Software Product Requirements and Software Project Requirements
- Functional Requirements
- Nonfunctional Requirements
- Technology Constraints
- Quality of Service Constraints
- Why Categorize Requirements This Way?
- System Requirements and Software Requirements
- Derived Requirements
- Software Requirements Activities

Requirements Elicitation

- Requirements Sources
- Common Requirements Elicitation Techniques

Requirements Analysis

- Basic Requirements Analysis
- Economics of Quality of Service Constraints
- Formal Analysis
- Addressing Conflict in Requirements

Requirements Specification

- Unstructured Natural Language Requirements Specification
- Structured Natural Language Requirements Specification
- Acceptance Criteria-based Requirements Specification
- Model-Based

Requirements Validation

- Requirements Reviews
- Simulation and Execution
- Prototyping

Requirements Management Activities

- Requirements Scrubbing
- Requirements Change Control
- Scope Matching

Practical Considerations

- Iterative Nature of the Requirements Process
- Requirements Prioritization
- Requirements Tracing
- Requirements Stability and Volatility

Software Requirements Tools

- Requirements Management Tools
- Requirements Modeling Tools
- Functional Test Case Generation Tools

8.3

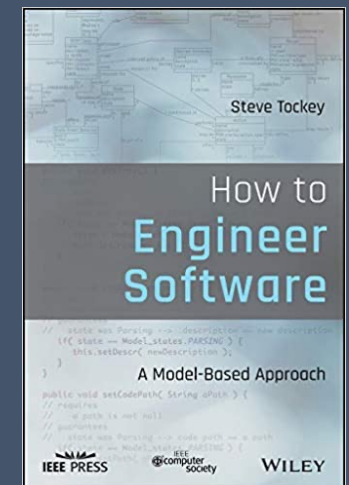
**Work breakdown item:
8.3. Functional test case generation tools**

Functional test case generation tools

- The more formally defined a requirements specification language is, the more likely it is that functional test cases can be at least partially derived mechanically.
- For example, converting BDD scenarios into test cases is not difficult.
- Another example involves state models.
 - ✓ Positive test cases can be derived for each defined transition.
 - ✓ Negative test cases can be derived from the state and event combinations that do not appear.

Functional test case generation tools

- A **process for deriving test cases from UML requirements models** can be found in the **book** *“How to Engineer Software”*.
- In the **general**, such tools can **only generate test case inputs**.
 - ✓ Determining an **expected result** is **not always possible**.
 - ✓ Additional **business domain expertise** might be **necessary**.



Questions?